

VIDEOINDEX

VideoIndex: architecture, milestones and work performed

Team catch-up report

A single document for new team members: the design as written (architecture, data model, pipeline, query layer, providers, SDK surfaces), the milestone plan and where it stands, the measured results so far, and a work log generated from the git history.

Generated from the repository by `scripts/report/team_report.py`
2026-09-12 · 95ca112

Contents

Executive summary	5
Where the project stands	5
What this document contains	5
Next steps	5
01. Overview	6
The problem	6
What VideoIndex is	6
Goals	6
Non-goals for v1	6
Who uses it	7
Guiding principles	7
Positioning	7
02. Architecture	8
Layers	8
Cargo workspace	9
Indexing data flow	10
Query data flow	11
Process model	11
Concurrency model	12
Deployment shapes	12
03. The Rust boundary	13
Short answer	13
What goes in Rust and why	13
What stays in Python and JavaScript	14
Binding strategy	14
The plugin bridge	15
Security	16
Realtime efficiency	16
What we give up	17
04. Data model	18
Time model	18
Entities	18
Storage trait	20
Embedded index directory layout	21
Full-text and vector search details	21
Schema versioning	21
05. Indexing pipeline	22
Stages	22
Operator contract	25
Scheduler	25
Caching	25
Budgets and policies	26
Progressive indexing	26
Failure handling	26

06. Query and agents	27
Retrieval	27
The agentic loop	27
Answers and citations	29
Multi-video and cross-video queries	30
Conversation state	30
MCP server	30
Performance targets	30
07. Model providers	31
Capability traits	31
Capabilities negotiation	31
Adapters	31
Provider configuration	32
Cross-cutting behavior in vi-providers	32
A/B evaluation support	33
Local versus remote decision guide	33
08. Evaluation	34
Benchmarks	34
Harness design	34
Metrics	35
Development dataset	35
Regression gates	36
09. SDK and APIs	37
Python	37
Node.js	37
CLI	38
Server (HTTP)	38
Configuration file	38
Stability policy	39
11. Deployment: videoindex.app on azuremc	40
What runs where	40
Subdomains	40
Disk layout	41
systemd units	41
Secrets	41
GPU detection	41
Getting videos onto azuremc	41
Observability	42
Backups	42
Rollout	42
Milestones and roadmap	43
M0. Skeleton and decode (first)	43
M1. Coarse index and search	43
M2. Fine index, agent, Python	44
M3. Node, server, MCP, chat app	44
M4. Evaluation	44
M5. Hosted mode	45

Later	45
Repository layout	45
Licensing	45
Open questions	45
M1 report: what was built and measured	46
What was built	46
Timings on this machine (12 vCPUs, one A100 80 GB)	46
Dev set results	49
What the design got wrong or left unclear	50
Open items carried into M2/M4	51
Work performed	52
Commits by type	52
Code by crate	52
Commit log by day	52
Decisions	55
Development machine	63
Hardware	63
Disks	63
Toolchain	63
GPU stack	64
Model files	65
Network	66
Existing deployment state	66
M0 re-verification on this machine (2026-09-11, release build)	66

Executive summary

VideoIndex turns long videos into a queryable knowledge base: an SDK and infrastructure framework whose core is Rust, with a Python binding, a CLI and (planned) a server with HTTP, SSE and MCP. Applications such as a video question-answering chat sit on top of the SDK.

Where the project stands

- **M0 (skeleton) and M1 (coarse index) are complete.** Every video that enters the system is decoded in a sandboxed worker, sampled at 1 fps, hashed, thumbnailed, cut into shots, transcribed (Silero VAD + Whisper large-v3 on the GPU), read for on-screen text (RapidOCR), and embedded (SigLIP for frames, bge-small for text). Hybrid search fuses BM25, text vectors and image vectors.
- **M2 is largely delivered.** Provider adapters for OpenAI-compatible servers, Anthropic and Gemini; an agent loop with tools, budgets, sessions and timestamp citations (`vi_ask`); the fine-pass operators (scenes, chapters, VLM descriptions, entities and events); a Python binding with operators and agent policies written in Python.
- **Measured on a 30-video, 36.6-hour dataset** (AI Engineer conference workshops and the Berkeley Agentic AI MOOC): retrieval hit@5 0.94 and MRR 0.72 on 72 questions; question answering 96.2% with citations on 52 questions at \$0.05 per question, against a 73.1% retrieval-only baseline. Coarse indexing runs at 56× real time on the GPU build.

What this document contains

Part I reproduces the design documents that define the system (overview, architecture, the Rust boundary, data model, indexing pipeline, query and agents, model providers, evaluation, SDK surfaces, deployment). Part II covers the milestones and the M1/M2 report with measurements. Part III is generated from the repository: the work log by day, code and test counts, the dated decisions made where the design was silent, and the facts about the development machine.

Next steps

1. Re-index the dataset with the CUDA build to measure the GPU end to end (about two hours).
2. A measured fine pass over the dataset (roughly \$60 at \$1.60 per hour of video with Claude Sonnet 5).
3. Programmatic comparison against Gemini's agentic video understanding on the same questions.
4. M3: Node binding, `vi-server` (HTTP, SSE, MCP) and the chat application at `videoindex.app`.

01. Overview

The problem

Long videos are the least searchable common media type. An hour-long conference workshop, a lecture series, a recorded meeting, or a security camera day holds information that today is reached only by scrubbing. Frontier video-language models can now answer questions about video, but sending an hour of frames to a model per question is slow, expensive, and non-repeatable. The recent shift, exemplified by Gemini's agentic video understanding, is to let a model decide *what to watch, at what rate, through which modality* instead of feeding every frame uniformly. Google reports up to 88% fewer tokens and up to 66% lower cost with higher accuracy from that approach.

That approach needs infrastructure the model does not provide: fast decoding and seeking, a persistent multimodal index, retrieval, budgets, caching, and a stable tool surface. VideoIndex is that infrastructure.

What VideoIndex is

VideoIndex is:

1. **An indexing engine.** It ingests long videos once and produces a persistent, portable knowledge base: segments, transcripts, on-screen text, visual descriptions, entities, events, and embeddings, each with timestamps and provenance.
2. **A query engine.** It answers questions over that knowledge base using hybrid retrieval plus an agentic loop that can go back to the pixels when needed, with timestamped citations.
3. **A framework.** Models, storage, and operators are swappable behind traits so that open-source and frontier models can be compared A/B on the same index.
4. **An SDK.** A Rust core with first-class Python and Node.js bindings, a CLI, and a server mode with HTTP, streaming, and MCP so any agent framework can use an index as a tool.

The analogy is LlamaIndex for documents: an open-source SDK that people run locally, with a hosted version available for those who do not want to run infrastructure.

Goals

- Index hour-scale videos at a cost and time that make batch indexing of hundreds of hours practical.
- Answer questions interactively, with first tokens within a couple of seconds and citations to exact timestamps.
- Work with any model backend: Gemini, Claude, OpenAI, vLLM or Ollama-hosted open models, and local ONNX models for lightweight perception.
- Make a video index a portable artifact: copy a directory and query it elsewhere.
- Be measurable: run LVBench, Minerva, and 1H-VideoQA end to end and report accuracy, tokens, cost, and latency per configuration.
- Keep the SDK independent of any application. The chat app is a consumer of the SDK.

Non-goals for v1

- Live stream ingestion (RTSP, HLS, WebRTC). The architecture leaves room for it, but v1 is files and URLs.
- Video editing, transcoding as a product, or generation.
- Training or fine-tuning models.

- Bindings beyond Python and Node.js. Other languages use server mode.
- A general-purpose vector database. Storage is embedded first, pluggable second.

Who uses it

- **SDK developers** building agentic applications who want video as a first-class retrievable source alongside documents.
- **Application developers** building the video QnA chat app or similar products on top.
- **Researchers** comparing sampling strategies and model backends on long-video benchmarks.
- **Hosted customers** who want the API without running decoders and indexes themselves.

Guiding principles

1. **Coarse to fine.** Index cheaply and broadly first, then spend model budget only where a query or a policy asks for it. Every expensive step must be justified by retrieval or an explicit request.
2. **Budgets are first class.** Every indexing job and query has token, cost, and time budgets. Running out of budget produces a partial, honest answer, never a hang.
3. **Provider agnostic.** No provider-specific type escapes the provider layer. The same index must be queryable with a different VLM than the one that built it.
4. **Provenance everywhere.** Every stored fact records the operator, provider, model version, and prompt that produced it. This is what makes A/B comparison and cache invalidation possible.
5. **Portable index.** The embedded backend produces a self-contained directory with a versioned manifest. Hosted backends implement the same storage trait.
6. **Untrusted input.** Video files, URLs, transcripts, and on-screen text are attacker-controlled data. Decoders are sandboxed and text derived from video is never treated as instructions.
7. **Rust for the machine, Python for the ideas.** Everything that touches bytes, threads, storage, or the network is Rust. Prompts, agent policies, and evaluation logic iterate in Python with Rust defaults. See [03-rust-boundary](#).

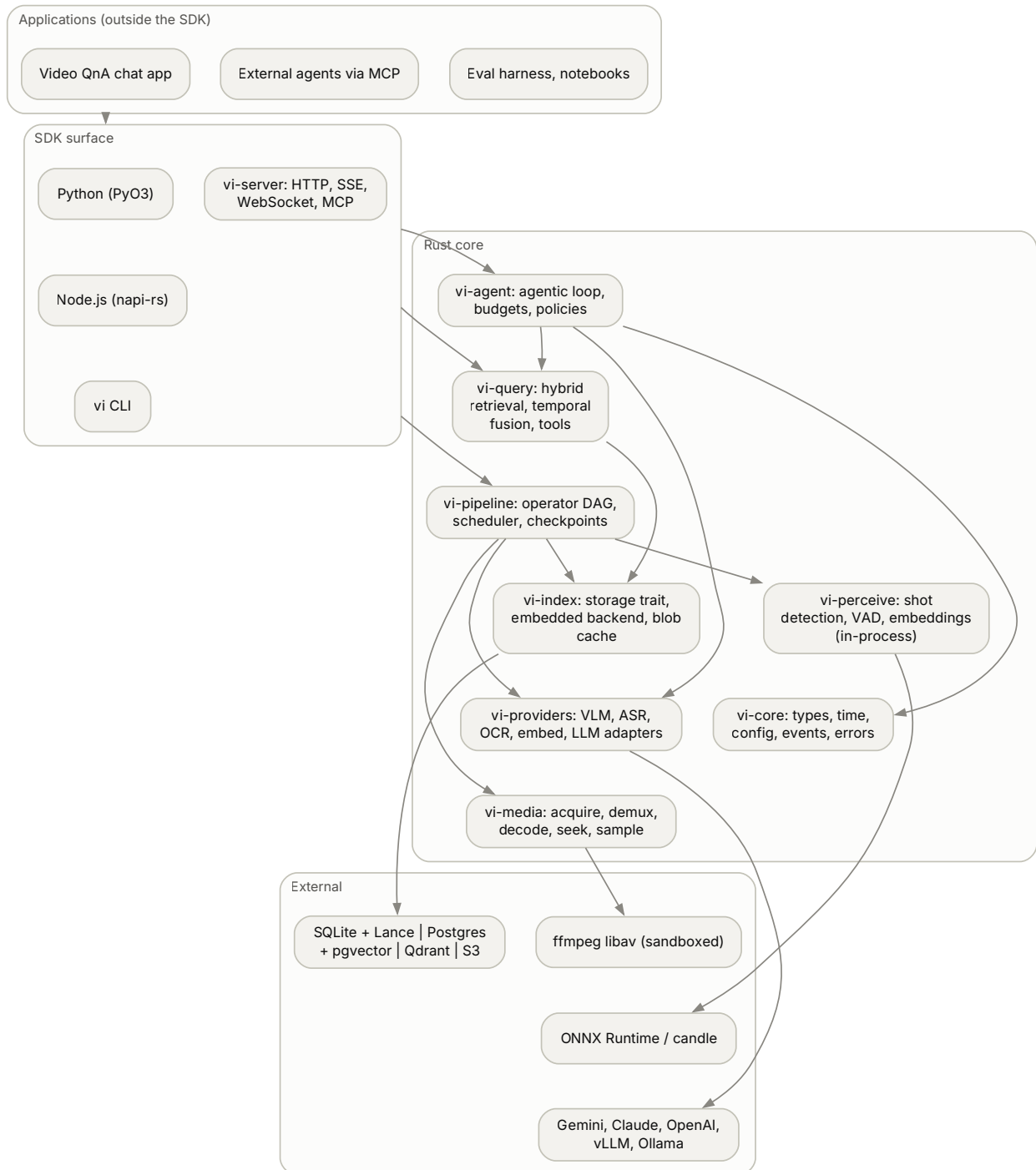
Positioning

	VideoIndex	Gemini agentic video	LlamaIndex	Twelve Labs-style APIs
Open source	Yes, SDK	No	Yes	No
Runs locally	Yes	No	Yes	No
Model choice	Any	Gemini only	Any LLM	Vendor models
Persistent index	Yes, portable	Per-request	Yes	Hosted only
Video native	Yes	Yes	Minimal	Yes
Agent tool surface	Native + MCP	Internal	Native	REST

VideoIndex adopts the agentic-video idea and makes it model-independent, persistent, and open.

02. Architecture

Layers



Cargo workspace

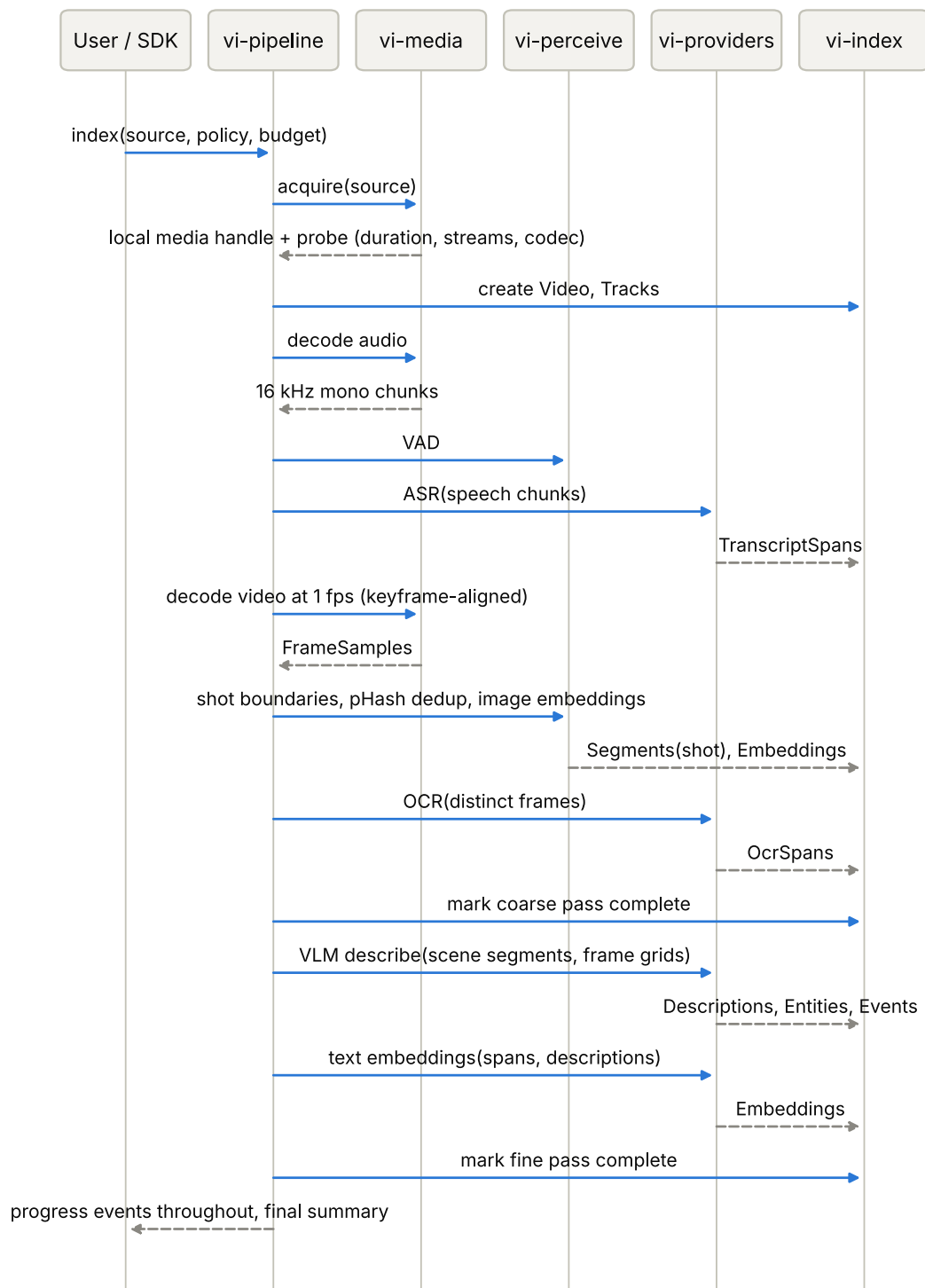
```

videoindex/
  Cargo.toml          workspace
  crates/
    vi-core/          shared types, time model, config, events, errors
    vi-media/          acquirers, ffmpeg demux/decode, hardware decode, seeking, sampling, tiling
    vi-perceive/       in-process operators: shot boundary, pHash, VAD, image/text embeddings
    vi-providers/      provider traits + adapters, rate limiting, retries, cost accounting
    vi-index/          storage trait, embedded backend (SQLite+FTS5, Lance/usearch), blob cache
    vi-pipeline/       operator DAG, scheduler, checkpoints, progress, caching
    vi-query/          retrieval, temporal fusion, tool primitives
    vi-agent/          default agentic loop, budgets, policy trait
    vi-server/         axum HTTP/SSE/WebSocket API, MCP server
    vi-cli/            `vi` binary
    vi-testkit/       test fixtures (synthetic video generated by ffmpeg at build time); dev-dependency
  only
  bindings/
    python/           Py03 + maturin → `videoindex` wheel
    node/             napi-rs → `@videoindex/core` package
  eval/              Python: benchmark loaders, runners, reports
  apps/
    chat/             demo QnA app (separate deployable, consumes SDK)
    site/             videoindex.app SDK site (later)
  docs/
  dataset/
  scripts/

```

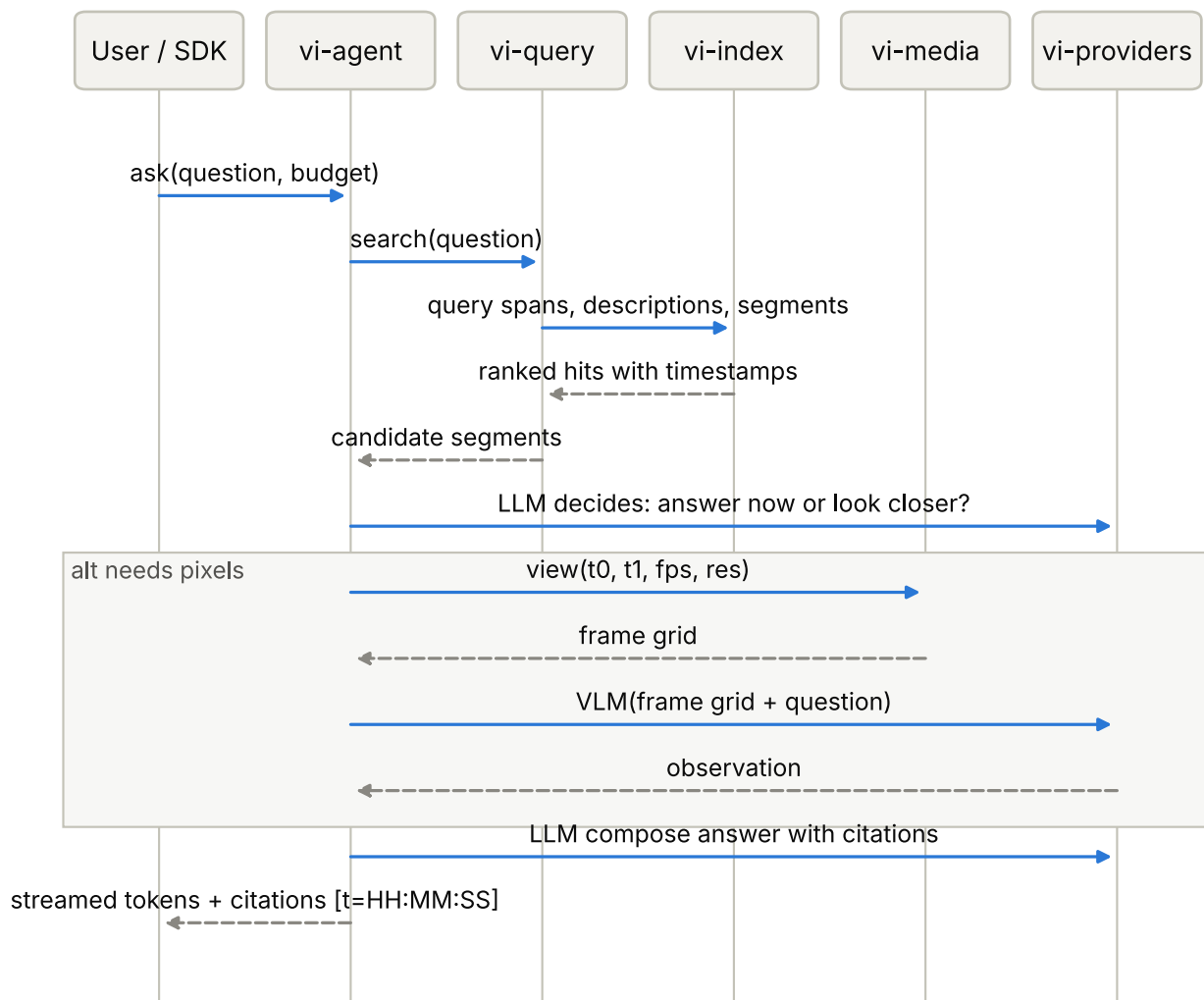
Crate responsibilities are described in the documents that follow. The one-line rule for placing code: if it needs frames, threads, storage, or sockets it goes in a `vi-*` crate. If it is a prompt, a policy, or an experiment it can start in Python and be ported to Rust once it stabilizes.

Indexing data flow



The pipeline is a DAG of operators, not a fixed sequence. The sequence above shows the default policy. Each arrow into `vi-index` is checkpointed, so a killed job resumes.

Query data flow



Retrieval is always tried first because it is cheap and cached. Decoding and VLM calls happen only when the agent decides the index does not contain the answer, and only within budget.

Process model

VideoIndex runs in three shapes from the same crates:

Shape	Entry	Use
Library	Python or Node binding	Embedded in a user's process. Pipeline and query run in the binding's tokio runtime, off the GIL / event loop.
CLI	<code>vi</code>	Batch indexing, scripting, eval.
Server	<code>vi-server</code>	Hosted or shared deployments. HTTP + SSE for answers, WebSocket for progress, MCP for agents. Multi-index, auth, quotas.

Decoding of untrusted media always runs in a child process (`vi-media` worker) with resource limits, regardless of shape. See [03-rust-boundary](#).

Concurrency model

- One tokio runtime per process. CPU-bound work (decode, embeddings, hashing) runs on a dedicated rayon pool or in the decode worker process; the async runtime never blocks.
- Provider calls are async with per-provider concurrency limits and token-bucket rate limits.
- Frames move as `Arc<FrameBuffer>` with a pixel format tag. No copies between decode, hashing, embedding, and tiling. Frames cross the Python boundary as NumPy arrays via the buffer protocol without a copy.
- Backpressure: the scheduler bounds in-flight frames per job so an hour of 4K video never sits in memory.

Deployment shapes

- **Local:** `pip install videoindex` or `npm install @videoindex/core`, index lives in a directory.
- **Self-hosted:** `vi serve` behind a reverse proxy, indexes on local disk or object storage.
- **Hosted (videoindex.app):** `vi-server` with the pluggable storage backends, multi-tenant, quotas, keys. See [11-deployment](#).

03. The Rust boundary

This document answers the question: how much of VideoIndex should be Rust, and how does it reach Python, Node.js, and other languages?

Short answer

Roughly 80% of the runtime code, and 100% of the code on the hot path, is Rust. Everything that touches bytes, frames, threads, storage, or sockets lives in the Rust core. Python and JavaScript hold the parts that change weekly: prompts, agent policies, evaluation logic, and applications. Python and Node get native bindings. Every other language gets the server.

What goes in Rust and why

Concern	In Rust	Why Rust specifically
Acquire, demux, decode, seek	Yes	Zero-copy frame buffers, hardware decoders, tight control of memory for hour-long inputs. Python decode loops (OpenCV, PyAV) copy frames and serialize on the GIL.
Frame sampling, tiling, hashing	Yes	CPU-bound and parallel. SIMD-friendly. Runs at thousands of frames per second per core.
In-process perception (shot detection, VAD, embeddings)	Yes	ONNX Runtime and candle have mature Rust bindings. No Python interpreter needed in the hosted server.
Provider HTTP clients	Yes	Connection pooling, retries, rate limits, streaming parsers, and cost accounting are shared by all bindings. One implementation, one set of bugs.
Storage and retrieval	Yes	SQLite, Tantivy/FTS5, Lance, and usearch all have native Rust APIs. Query latency is dominated by this layer and must be predictable.
Scheduler, checkpoints, budgets	Yes	Structured concurrency with tokio. Correct cancellation and backpressure are hard to retrofit into Python.
Default agentic loop and tools	Yes	The loop is small, and having it in Rust means the CLI, server, and MCP all behave identically without a Python runtime. The policy is a trait so Python can override it.
Server, MCP	Yes	axum plus the official Rust MCP SDK. A single static binary deploys to azuremc.

The core reasons, in order of importance for this project:

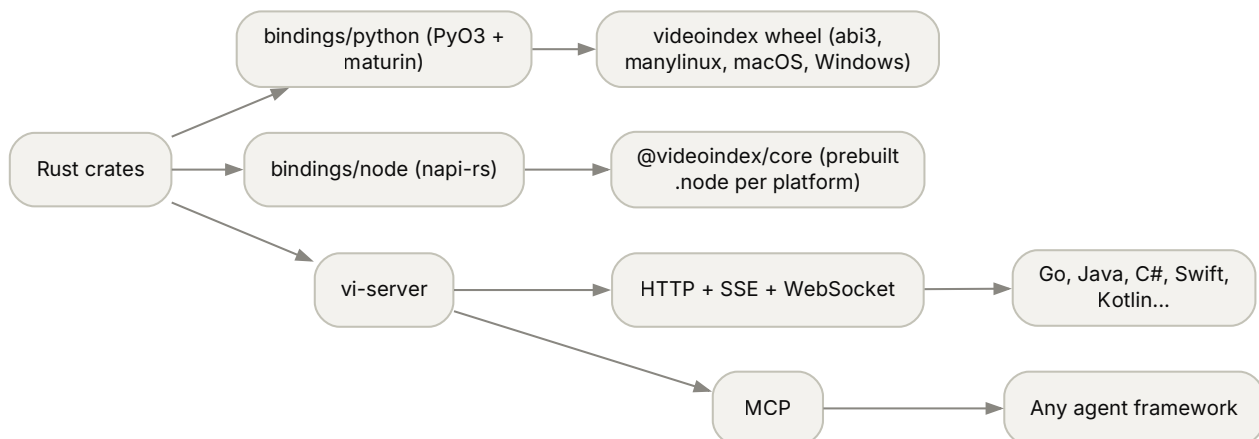
- 1. Throughput on decode and sampling.** Indexing an hour of 1080p video means decoding tens of thousands of frames. In Rust with libav and hardware decode this is a few minutes on a laptop; the same in Python with per-frame copies is often the slowest part of the whole pipeline.
- 2. No GIL.** Interactive queries need retrieval, decoding of a small window, and a VLM call to overlap. Rust does this in one process with real threads. The Python binding releases the GIL for every call into the core.
- 3. Memory safety on untrusted input.** Video containers and codecs are a historic source of memory-corruption bugs. Rust does not fix ffmpeg's C code, but it lets us isolate it: the decoder runs in a separate sandboxed process and the rest of the system is safe Rust. A Python process embedding ffmpeg cannot offer that boundary cheaply.
- 4. One core, many surfaces.** The same crates power the Python wheel, the Node package, the CLI, the server, and later WASM and mobile. Writing the core in Python would mean a Node port or a mandatory server for JavaScript users.

5. **Predictable latency.** No garbage collector pauses in the query path, and no interpreter startup in the CLI.
6. **Single binary deployment.** `vi-server` ships as one static binary plus ffmpeg. The hosted version does not need a Python environment.

What stays in Python and JavaScript

Concern	Language	Why not Rust
Prompt templates, few-shot examples	Python/JS, with Rust defaults	Iteration speed. Prompts change daily during development. Rust ships defaults so the CLI and server work without them.
Experimental agent policies	Python, via the <code>Policy</code> trait bridge	Researchers test new "what to watch next" strategies in notebooks. Winners get ported to Rust.
Evaluation harness	Python (<code>eval/</code>)	Dataset loaders, pandas, plotting, and benchmark scripts are a Python ecosystem. The harness calls the SDK; the SDK does not know about benchmarks.
Custom operators using PyTorch or HF models	Python, as a callback operator or a sidecar	The ML ecosystem is Python. VideoIndex never tries to load PyTorch in-process; it calls out.
Chat app, SDK website	TypeScript	Web applications. They consume the Node binding or the HTTP API.

Binding strategy



Python: PyO3 + maturin

- Distributed as `abi3` wheels, one per platform, not per Python version. Python 3.10 and newer.
- Async: the core's tokio runtime is owned by the binding module. Python `async def` methods return awaitables via `pyo3-async-runtimes`. Synchronous methods also exist and release the GIL.
- Frames: exposed as NumPy arrays through the buffer protocol over the core's `Arc<FrameBuffer>`, no copy. Lifetimes are enforced by holding the `Arc` in the Python object.
- Streaming answers: an `AsyncIterator` of typed events `( token , tool_call , citation , done )`.
- Callbacks into Python (custom operators, custom policies) go through a `PyOperator` adapter that implements the Rust `Operator` trait, acquires the GIL only for the call, and runs on a dedicated thread so a slow Python operator never blocks the runtime.
- Typed stubs `( .pyi )` generated at build time so IDEs and type checkers see the API.

Node.js: napi-rs

- Prebuilt native modules for linux-x64, linux-arm64, darwin-arm64, darwin-x64, win32-x64. Node 20 and newer.
- Async methods return Promises. Streams are `AsyncIterable`.
- Frames are `Uint8Array` views over external buffers, no copy.
- Callbacks into JS use threadsafe functions with the same isolation rule as Python.
- TypeScript definitions generated by napi-rs.

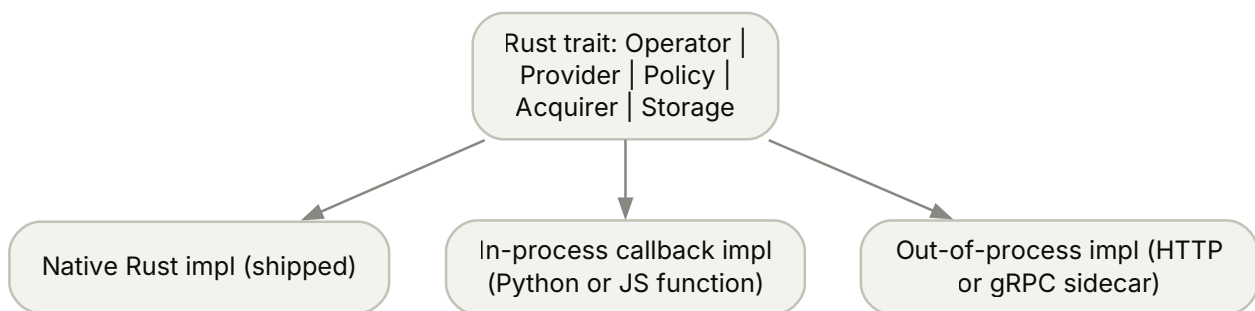
Everything else: server mode

Go, Java, C#, Swift, Kotlin, Ruby, and any agent framework use `vi-server`. The HTTP API is OpenAPI-described so clients can be generated. MCP exposes the same tools the internal agent uses, so an external agent can drive an index exactly as the built-in loop does.

A stable C ABI via UniFFI is deliberately deferred. It becomes worthwhile only if on-device mobile indexing is pursued; until then it is maintenance without users.

The plugin bridge

Every extension point is a Rust trait with three kinds of implementation:



- **Native** implementations cover everything needed for the CLI and server to work alone.
- **Callback** implementations let a Python or JS function stand in for a trait method. Good for prompts, policies, and light custom logic. The bridge marshals typed structs (`serde` on the Rust side, `dataclasses` / `TypeScript` interfaces on the other). The language-neutral half lives in `vi_pipeline::callback`: items become JSON objects with a `kind` tag (pixels and audio samples are attached by the binding as native arrays), and rows come back as JSON (`CallbackRow`: `transcript span`, `OCR span`, `shot`, `scene`, `chapter`, `description`) that the core turns into stored rows with ids and provenance before emitting them downstream. `Scheduler::register_operator` adds the operator under a name policies can use, replacing a built-in one of the same name; `JobOptions::inline_policy` runs a policy the caller assembled at runtime. The Python binding's `PyOperator` and `PyPolicy` are thin: attribute reading, dict conversion, and the GIL-holding call on a blocking thread.
- **Sidecar** implementations are how heavy Python models plug in without loading PyTorch into the VideoIndex process: run Qwen-VL under vLLM and point the OpenAI-compatible provider at it, or expose a custom operator as a tiny HTTP service. The hosted version uses only native and sidecar implementations.

Security

Threat model: video files, URLs, subtitles, transcripts, and on-screen text are attacker-controlled. Model outputs are untrusted. The VideoIndex process may hold API keys.

- **Sandboxed decode.** `vi-media` spawns a worker process for demux and decode. On Linux it runs under `seccomp` and a restricted user with memory, CPU, and file-descriptor limits; on macOS under `sandbox-exec`; on all platforms with a wall-clock timeout per job. The parent only receives frame buffers over shared memory and metadata over a length-prefixed protocol. A crash in `libav` kills the worker, not the index.
- **Acquire allowlist.** URL acquirers refuse private address ranges by default and enforce size and time limits. The YouTube acquirer shells out to `yt-dlp` with a fixed argument set and never passes user input as flags.
- **Prompt-injection defense.** Transcripts and OCR text are data. They enter prompts inside delimited, labeled blocks with provenance, and the agent's tools have no side effects beyond reading the index and decoding frames. Nothing the model says can write to the index or call the network except through the typed provider layer.
- **Provenance and integrity.** The index manifest is schema-versioned and carries content hashes of the SQLite and vector files. Optional signing lets a hosted service reject tampered indexes.
- **Secrets.** Provider keys come from environment or a key file, never from the index directory, and are redacted from logs and error messages.
- **Dependency hygiene.** `cargo-deny` and `cargo-audit` in CI, minimal `unsafe` confined to the `ffmpeg` FFI and buffer protocol shims, each block documented.

Realtime efficiency

The interactive path is: question arrives, retrieval returns candidates, maybe a small decode, maybe one VLM call, stream the answer. Targets on a modest server (8 cores, no GPU): retrieval under 50 ms, decode of a 10-second window at 2 fps under 300 ms, first streamed token under 2 s excluding provider latency.

Techniques, all in the Rust core:

- **Hardware decode** through `libav` (VideoToolbox on macOS, NVDEC on NVIDIA, VAAPI on Linux), with software fallback.
- **Keyframe-aware seeking.** Seek to the nearest keyframe before `t0`, decode forward, drop frames before `t0`.
- **Frame grids.** Compose N frames into one tiled image with timestamp overlays so a VLM call sees a time range for the token cost of one image.
- **Perceptual dedup.** Consecutive near-identical frames (slides, static shots) collapse via `pHash` before any model sees them.
- **Content-addressed caches.** Every operator output is keyed by input content hash, operator version, provider, model, and prompt hash. Re-indexing with a new VLM re-runs only the VLM step.
- **Progressive index.** The coarse pass (transcript, shots, OCR, embeddings) completes first and unlocks queries. The fine pass (VLM descriptions) refines in the background.
- **Budget-aware scheduler.** Each query and job carries a budget. The scheduler prefers cached and cheap operators and stops issuing new model calls when the budget is spent.
- **Bounded memory.** In-flight frames per job are capped. Thumbnails, not full frames, are persisted.

What we give up

- **Build complexity.** ffmpeg and ONNX Runtime are native dependencies. We vendor prebuilt static libraries per platform in CI and provide a `--features external-ffmpeg` path for distro builds.
- **Contributor pool.** Fewer people write Rust than Python. Mitigation: the extension points are Python-friendly, and the eval harness and prompts are Python.
- **Iteration speed in the core.** Compile times and type ceremony. Mitigation: start experiments in Python via the bridge, port when stable.

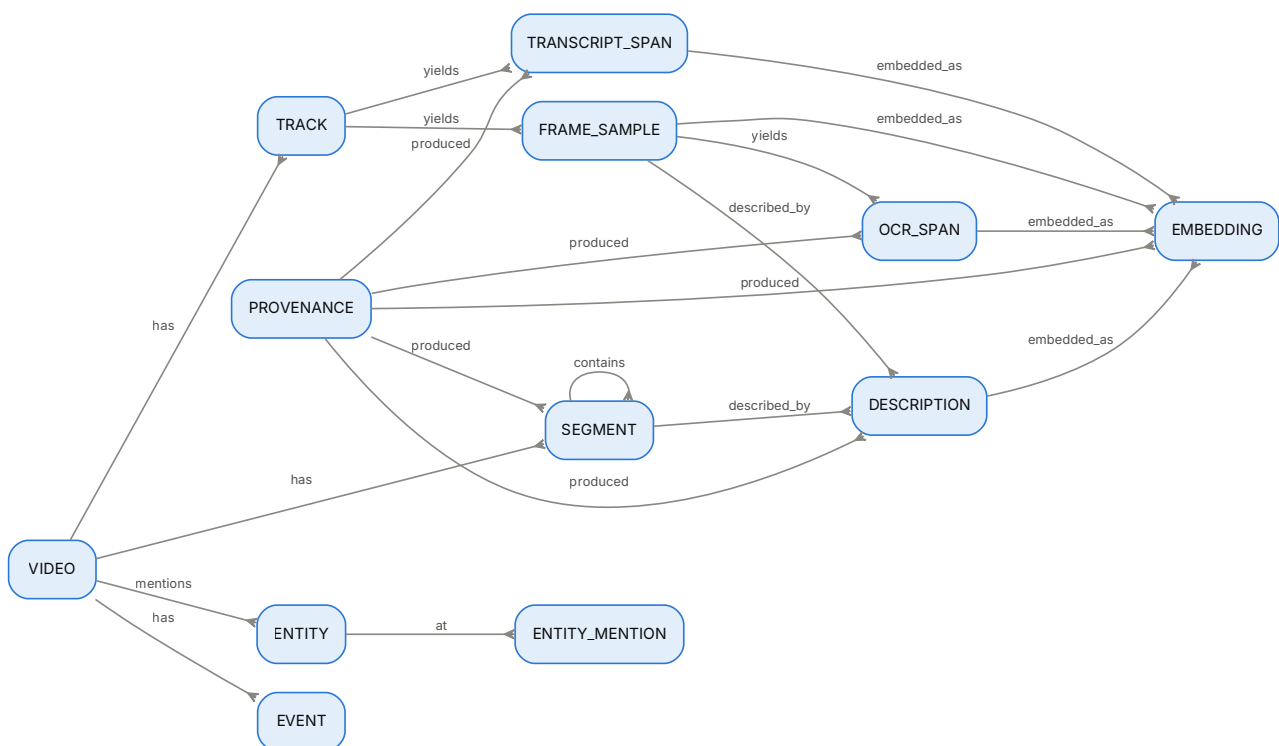
04. Data model

Time model

Every timestamp in the index is a `Timestamp { num: i64, den: u32 }` rational in the source Track's timebase, plus a derived `f64` seconds column for indexing and display. Rationals avoid drift over hour-long videos with odd frame rates (29.97, 23.976). Wall-clock time, when known from container metadata, is stored once per Video as `start_wallclock` so events can be placed on a calendar without touching every row.

Ranges are half-open: `[t0, t1)`.

Entities



Video

Field	Type	Notes
id	ULID	Stable across re-indexing of the same content
source_uri	text	Original location
content_hash	blake3	Of the media file; drives caching and dedup
title, description, channel, published_at	text/ts	From container or yt-dlp metadata
duration	Timestamp	
start_wallclock	ts, nullable	
probe	JSON	ffprobe-equivalent output
index_state	enum	<code>acquired</code> , <code>coarse</code> , <code>fine</code> , <code>failed</code>

Track

`id, video_id, kind (video|audio|subtitle), stream_index, codec, timebase, width, height, fps, sample_rate, channels, language.`

Segment

The retrieval unit. A hierarchy: `shot` (visual cut boundaries), `scene` (grouped shots, typically 20 s to 3 min), `chapter` (from chapters metadata, slide titles, or topic shifts, typically minutes). Every level spans the whole video with no gaps.

`id, video_id, level, parent_id, t0, t1, keyframe_sample_id, title (nullable), summary (nullable), provenance_id.`

FrameSample

`id, track_id, t, pts, is_keyframe, phash (u64), thumbnail_blob (nullable), width, height.` Only sampled frames exist here. Full frames are never stored; they are re-decoded on demand.

TranscriptSpan

`id, track_id, t0, t1, text, speaker (nullable), language, confidence, words (JSON, word-level timings when available), provenance_id.`

OcrSpan

`id, frame_sample_id, t, text, bbox (x, y, w, h normalized), confidence, provenance_id.`

Consecutive identical OCR text across frames is collapsed into a range on a derived view, not duplicated in storage.

Description

Text a VLM produced about a Segment or a FrameSample.

`id, target_kind (segment|frame), target_id, kind (caption|summary|qa|structured), text, structured (JSON, nullable), provenance_id.` Several Descriptions can exist for the same target from different providers, which is what enables A/B evaluation on one index.

Entity and EntityMention

Entity: `id, video_id, kind (person|object|text|place|concept), name, canonical_name, attributes (JSON).` EntityMention: `entity_id, t0, t1, source_kind (transcript|ocr|description), source_id, confidence.`

Event

`id, video_id, t0, t1, text, participants (entity ids), provenance_id.` Extracted from Descriptions and transcript by an LLM operator.

Embedding

`id, target_kind, target_id, model, dim, vector.` Stored in the vector store, referenced from SQLite by `(target_kind, target_id, model).` Multiple models can coexist.

Provenance

`id, operator, operator_version, provider, model, model_version, prompt_hash, params (JSON), created_at, cost_usd, tokens_in, tokens_out, latency_ms.` Every derived row points at exactly one Provenance. Cost roll-ups per video and per configuration are a single aggregate over this table.

Storage trait

```
#[async_trait]
pub trait Storage: Send + Sync {
    // metadata
    async fn put_video(&self, v: &Video) →
    Result<>; // upsert; VideoId is stable per content hash
    async fn get_video(&self, id: VideoId) → Result<Option<Video>>;
    async fn find_video_by_hash(&self, content_hash: &str) → Result<Option<Video>>;
    async fn list_videos(&self) → Result<Vec<Video>>;
    async fn set_index_state(&self, id: VideoId, state: IndexState) → Result<>;
    async fn put_tracks(&self, t: &[Track]) → Result<>;
    async fn tracks(&self, video: VideoId) → Result<Vec<Track>>;
    async fn put_segments(&self, s: &[Segment]) → Result<>;
    async fn put_frame_samples(&self, s: &[FrameSample]) → Result<>;
    async fn update_frame_phash(&self, updates: &[(FrameSampleId, u64)]) → Result<>;
    async fn update_frame_thumbnail(&self, updates: &[(FrameSampleId, BlobKey)]) → Result<>;
    async fn delete_frame_samples(&self, track: TrackId) → Result<u64>;
    async fn frame_samples(&self, track: TrackId, range: Option<TimeRange>) → Result<Vec<FrameSample>>;
    async fn put_spans(&self, s: &[Span]) → Result<>; // transcript + ocr
    async fn delete_spans_by_operator(&self, track: TrackId, operator: &str) → Result<u64>; // re-runs replace
    // their own output
    async fn spans_by_operator(&self, video: VideoId, operator: &str) →
    Result<Vec<Span>>; // replay of cached stages
    async fn put_descriptions(&self, d: &[Description]) → Result<>;
    async fn put_embeddings(&self, e: &[Embedding]) → Result<>;
    async fn put_provenance(&self, p: &Provenance) → Result<ProvenanceId>;
    async fn get_embeddings(&self, model: &str, targets: &[(TargetKind, String)]) → Result<Vec<Option<Vec<f32>>>>;
    >>;
    async fn descriptions(&self, video: VideoId) → Result<Vec<Description>>;
    async fn put_entities(&self, e: &[Entity], m: &[EntityMention]) → Result<>;
    async fn entities(&self, video: VideoId) → Result<Vec<Entity>>;
    async fn put_events(&self, e: &[Event]) → Result<>;
    async fn events(&self, video: VideoId) → Result<Vec<Event>>;
    async fn delete_extractions(&self, video: VideoId) → Result<u64>;
    // search
    async fn text_search(&self, q: &TextQuery) → Result<Vec<Hit>>; // BM25 / FTS
    async fn vector_search(&self, q: &VectorQuery) → Result<Vec<Hit>>;
    async fn time_window(&self, video: VideoId, t0: Ts, t1: Ts, kinds: &[Kind]) → Result<Window>;
    // blobs
    async fn put_blob(&self, key: &BlobKey, bytes: Bytes) → Result<>;
    async fn get_blob(&self, key: &BlobKey) → Result<Option<Bytes>>;
    // sessions (agent conversations, TTL)
    async fn put_session(&self, id: &str, state: &serde_json::Value, ttl_secs: u64) → Result<>;
    async fn get_session(&self, id: &str) → Result<Option<serde_json::Value>>;
    // jobs
    async fn checkpoint(&self, job: JobId, state: &JobState) → Result<>;
    async fn load_checkpoint(&self, job: JobId) → Result<Option<JobState>>;
    async fn list_jobs(&self) → Result<Vec<JobState>>;
    // maintenance
    async fn manifest(&self) → Result<Manifest>;
    async fn stats(&self) → Result<IndexStats>; // sizes and per-video counts for `vi
    status`
    async fn compact(&self) → Result<>; // VACUUM + refresh manifest hashes
}
```

Implementations:

Backend	Metadata + FTS	Vectors	Blobs	Use
Embedded (default)	SQLite + FTS5	Flat memory-mapped files per model (exact search); Lance or usearch later for approximate search	Files under <code>blobs/</code>	Local SDK, CLI, single-node server
Postgres	Postgres + tsvector	pgvector	S3/GCS/R2	Hosted, multi-tenant
Qdrant	Postgres or SQLite	Qdrant	S3/GCS/R2	Hosted at larger scale

Only the embedded backend ships in v1. The trait exists from day one so the pipeline and query layers never touch SQLite directly. The trait is implemented in `vi-index`; the record types live in `vi-core::model` so `vi-media` and `vi-pipeline` share them without depending on the storage crate.

Writes to parent tables (videos, tracks, frame samples, segments) are upserts. `INSERT OR REPLACE` would delete and re-insert the row and the `ON DELETE CASCADE` constraints would silently drop every child row (see `DECISIONS.md`).

Embedded index directory layout

```
myindex.vidx/
manifest.json      schema_version, created_by, index ids, content hashes, optional signature
meta.sqlite       all tables above, FTS5 virtual tables for spans and descriptions
vectors/
  <model-name>.vec  normalised f32 rows, append-only, memory-mapped for exact search
  <model-name>.meta per row: embedding id, video id, target kind, alive flag
blobs/
  ab/cd/abcdef...  content-addressed: thumbnails (WebP), audio chunks (Opus), frame grids
cache/
  operators/       operator output cache keyed by (content_hash, operator, version, provider, model,
prompt_hash)
jobs/
  <job-id>.json    checkpoints
```

Properties: - Copy the directory anywhere and open it. No absolute paths inside. - `manifest.json` is the only file a reader must parse before deciding whether it can open the index. Unknown newer schema versions are refused with a clear error; older ones are migrated in place with a backup. - `cache/` and `blobs/thumbnails` are safe to delete; they regenerate on demand from the source media if it is still reachable. - Sizes: roughly 30 to 80 MB per hour of video, dominated by thumbnails and embeddings. Tunable by thumbnail resolution and sampling rate.

Full-text and vector search details

- FTS5 with the `unicode61` tokenizer and prefix indexes for spans and descriptions. BM25 ranking. Tantivy is an alternative if FTS5 ranking quality proves limiting; the trait hides the choice.
- Vectors live in flat per-model files searched exactly (brute force, parallel); the `embeddings` table maps each row back to its target and the `.meta` file carries the video id and target kind so filters apply before scoring. An approximate index (Lance IVF-PQ or usearch HNSW) can replace the file behind the trait when tables grow past a few million rows.
- Temporal fusion, described in [06-query-and-agents](#), happens above the storage layer.

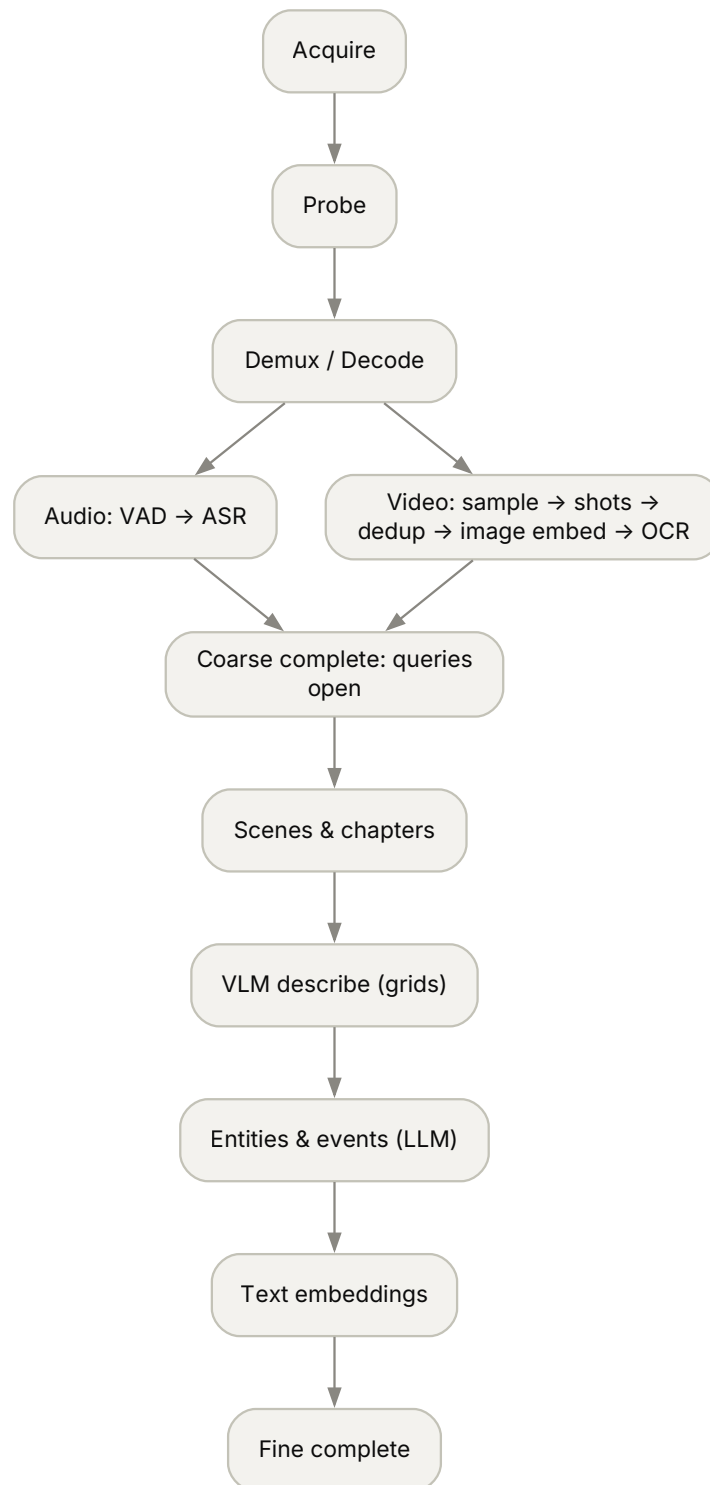
Schema versioning

The schema version is a single integer in `manifest.json` and in a `schema_meta` table. Migrations are Rust functions registered in order. Every migration is tested against fixture indexes produced by the previous version.

05. Indexing pipeline

Indexing turns a Source into a populated Index. It is a DAG of typed Operators run by a scheduler under a budget, with every output checkpointed and cached.

Stages



Acquire

The `Acquirer` trait resolves a `Source` to a local, seekable media file plus metadata.

Acquirer	Handles	Notes
<code>LocalFile</code>	paths	Zero-copy open
<code>ObjectStore</code>	<code>s3://</code> , <code>gs://</code> , <code>r2://</code> , <code>az://</code>	Via the <code>object_store</code> crate. Ranged reads allow probing without full download; full download for decode.
<code>Http</code>	<code>https://.../file.mp4</code>	Size and time limits, resumable, private-range blocked
<code>YtDlp</code>	YouTube, playlists, other sites yt-dlp supports	Shells out to <code>yt-dlp</code> with a fixed argument list: best video up to 720p plus best audio, merge to MP4, write <code>.info.json</code> , subtitles and auto-captions if present, download archive for idempotence. Playlists expand into one <code>Source</code> per entry. Absence of <code>yt-dlp</code> is a clear error with install hint. YouTube blocks most datacenter IPs, so on servers the usual path is files downloaded elsewhere and transferred; <code>LocalFile</code> imports the <code>yt-dlp</code> <code>.info.json</code> and subtitle sidecars next to a file so those transfers keep title, chapters, and captions.

Acquired media lands in a content-addressed media cache (`/data/videoindex/videos/<hash>.mp4` on servers; a configurable directory locally). Existing subtitles from acquisition are imported as a `subtitle Track` and can seed the transcript, saving ASR cost when they are human-authored.

Probe

libav-based probe: duration, streams, codecs, frame rate, resolution, keyframe interval estimate, chapter markers, embedded metadata. Stored on the `Video` record. Decides the decode plan (hardware or software, target sampling rate, whether audio exists).

Demux and decode

Runs in the sandboxed `vi-media` worker. Two independent streams:

- **Audio:** decoded and resampled to 16 kHz mono PCM, chunked at 30 s with 1 s overlap, aligned to VAD boundaries when available.
- **Video:** decoded at a sampling policy's rate (default 1 fps for the coarse pass), keyframe-aligned seeking so scrubbing an hour does not decode the full stream. Frames are delivered as `Arc<FrameBuffer>` (RGB24 or NV12) with PTS.

Hardware decode is auto-detected: `VideoToolbox`, `NVDEC`, `VAAPI`, else software. Decoding falls back to software on any hardware error for that job.

Audio operators

Operator	Where	Output
<code>Vad</code>	in-process (Silero via ONNX)	speech ranges
<code>Asr</code>	provider (Whisper via OpenAI-compatible or local server, Gemini, Deepgram-style)	<code>TranscriptSpans</code> with word timings when the provider gives them
<code>Diarize</code>	provider, optional	speaker labels on spans

Only speech ranges are sent to ASR. An hour of lecture with 40 minutes of speech costs 40 minutes of ASR. `Vad` decodes the whole track, scores it in one batched pass, merges speech across pauses under `models.vad.min_silence_ms`, drops runs under `min_speech_ms`, pads by `pad_ms`, and splits runs longer

than `max_segment_secs` at their quietest window. Each `SpeechRange` item carries its PCM so `Asr` never re-decodes. `Asr` sends `models.asr.concurrency` requests at a time through the `asr` role, splits the returned segments on word timings into spans of about `span_secs`, and writes one `Provenance` row per request.

Video operators

Operator	Where	Output
<code>Sample</code>	in-process	FrameSamples at policy rate
<code>ShotBoundary</code>	in-process (HSV histogram distance + edge change ratio, adaptive threshold)	shot Segments
<code>PHash</code>	in-process	64-bit perceptual hash per FrameSample; consecutive frames within Hamming distance 6 collapse for downstream operators
<code>ImageEmbed</code>	in-process (SigLIP or CLIP via ONNX) or provider	Embeddings for distinct FrameSamples
<code>Ocr</code>	provider (VLM with OCR prompt, PaddleOCR sidecar, cloud OCR) or in-process (RapidOCR ONNX)	OcrSpans for distinct frames whose text-likelihood heuristic fires
<code>Thumbnail</code>	in-process	WebP thumbnail per FrameSample at 320 px

Coarse complete

Once the audio and video operators above finish, the Video is marked `coarse`. Transcript search, OCR search, image-embedding search, and shot navigation all work. For lectures and talks, most questions are answerable here without a VLM.

Scene and chapter grouping

- **Scenes:** shots grouped by image-embedding similarity and transcript continuity, targeting 20 s to 3 min.
- **Chapters:** from container chapter markers or yt-dlp chapters when present; otherwise topic segmentation over the transcript (TextTiling-style over sentence embeddings) combined with slide-title OCR changes.

VLM describe

For each scene, compose a frame grid (default 3×3 of dedup'd keyframes with timestamp labels) and ask the VLM for a structured description: what is shown, on-screen text, people or objects, actions, and a one-line summary. Providers with native video input receive the clip directly instead of a grid, controlled by capability negotiation in [07-model-providers](#). The transcript for the scene is included in the prompt as labeled data so the description is grounded in speech.

Default budget: one VLM call per scene. Policies can request denser coverage for sections with high visual change or on demand from a query.

Entities and events

An LLM operator reads scene descriptions plus transcript in sliding windows and emits Entities, EntityMentions, and Events with time ranges. Entities are canonicalized across the video by name and embedding similarity.

Text embeddings

All TranscriptSpans, OcrSpans, Descriptions, and Segment summaries are embedded with the configured text embedding model. Spans are embedded in windows of roughly 30 s with overlap so retrieval hits carry enough context.

Operator contract

```
#[async_trait]
pub trait Operator: Send + Sync {
    fn id(&self) → &'static str;           // e.g. "shot_boundary"; also the stage name in checkpoints
    fn version(&self) → u32;               // bump when output semantics change
    fn inputs(&self) → &[InputKind];       // what it consumes
    fn outputs(&self) → &[OutputKind];     // what it produces
    fn cost_estimate(&self, input: &InputSummary) → CostEstimate;
    async fn run(&self, ctx: &OpContext, input: OpInput) → Result<OpOutput>; // one input item per call
    async fn finish(&self, ctx: &OpContext) → Result<OpOutput> { Ok(Default::default()) } // after the last
input
}
```

The scheduler derives the DAG from `inputs` and `outputs`. Adding an operator never requires editing the scheduler. `OpContext` gives access to providers, storage and the blob store, the budget, a cancellation token, progress reporting, and `emit`: operators stream each output item through `ctx.emit(item)` rather than returning a `Vec`, so an hour of frames never sits in memory and backpressure from bounded channels reaches the decoder. `finish` runs once after the last input so operators can flush batched storage writes. Input kinds are one enum (`ItemKind: Media, Frame, Hashed, Thumbnail, AudioChunk, ...`); every job starts by feeding the single `Media` item to the operators that declare it as input.

Scheduler

- Builds the DAG for a job from the chosen `IndexPolicy` (which operators, which providers, sampling rates).
- Runs operators as tokio tasks, with CPU-bound ones dispatched to the rayon pool and decode to the worker process.
- Bounded channels between operators provide backpressure; at most N frames are in flight per job. N is the decode worker's shared-memory slot count (`media.worker.max_in_flight_frames`), and channel capacities are derived from it so a slow consumer stalls the decoder instead of deadlocking it.
- Provider calls go through per-provider semaphores and rate limiters. Batched where the provider supports it.
- Every operator output is written to storage and the job checkpoint (`jobs/<job-id>.json`) is updated when a stage starts, finishes, or fails. A completed stage also leaves a marker in the operator output cache (below). Re-running a job plans each stage as *skip* (cached, no running consumer needs its items), *replay* (cached, a consumer needs its items and the operator can re-emit them from storage), or *run* (not cached, or a producer runs and so its inputs changed). `sample` cannot replay, so anything needing frames re-decodes; text stages (`subtitle_import`, `asr`, `ocr`) and `shot_boundary` replay.
- Cancellation propagates through the DAG in under a second; partial results remain queryable.
- Emits `Progress` events: stage, fraction complete, cost so far, ETA.

Caching

Operator outputs are cached under a key of (`input content hash, operator id, operator version, provider, model, model version, prompt hash, params hash`); in the embedded backend the outputs are the index rows themselves and the cache is a marker file per key under `cache/operators/`.

Consequences:

- Re-running with a different VLM re-runs only the VLM stage and downstream stages that depend on it.

- Two Videos with identical content share every cached output.
- The eval harness can sweep providers over the same 100 hours of video without re-decoding anything.

Budgets and policies

An `IndexPolicy` is a named configuration:

```
[policy.lecture_default]
sample_fps = 1.0
coarse = ["subtitle_import", "vad", "asr", "sample", "shot_boundary", "phash", "thumbnail", "image_embed",
"ocr", "text_embed"]
fine = ["scenes", "chapters", "vlm_describe", "entities_events"]
vlm_grid = "3x3"
max_cost_usd_per_hour = 2.0
max_wallclock_per_hour = "20m"

[policy.coarse_only]
fine = []
```

`default_policy = "auto"` (the default) picks `coarse_only` when the `asr`, `ocr`, `image_embed` and `text_embed` roles are bound and `coarse_local` (no provider-backed operators) otherwise. `text_embed` sits in the coarse pass because hybrid search needs span vectors; the fine pass adds the VLM and LLM stages in M2.

When a budget is exhausted the job stops issuing new provider calls, finishes writing what it has, marks the Video with the reached state, and reports which stages were skipped. `max_wallclock_per_hour` accepts `20m`, `1h30m`, `90s`; `0` for either limit means unlimited. The ceiling is the per-hour figure times the video's duration in hours (at least one minute's worth).

Progressive indexing

Indexing and querying can overlap. A query against a Video in `coarse` state uses what exists and reports the index state in its response so the caller can decide to wait, proceed, or trigger a targeted fine pass for the segments the query touched. The agent's `describe(t0, t1)` tool is exactly such a targeted fine pass: it runs the VLM operator for one range and stores the result, so the index improves with use.

Failure handling

- Corrupt or truncated media: probe reports what is decodable; indexing proceeds over the readable range and records the gap.
- Provider errors: retried with backoff and jitter; after the retry budget, the operator marks the affected inputs as `failed` with the error and the job continues. A summary lists failed ranges.
- Worker crash: the job restarts the worker and resumes from checkpoint; three crashes on the same range mark that range failed.

06. Query and agents

Two entry points share one engine:

- `search(...)`: deterministic retrieval, returns ranked timestamped hits. No model call unless the caller asks for reranking.
- `ask(...)`: the agentic loop. Uses `search` first, then decides whether to look at pixels, then composes an answer with citations. Streams.

Retrieval

Hybrid search

For a query string, `vi-query` runs in parallel:

1. **BM25** over TranscriptSpans, OcrSpans, Descriptions, and Segment summaries.
2. **Text vector** search over the same, using the configured text embedding model.
3. **Image vector** search over FrameSample embeddings, using the text tower of the same image-text model (SigLIP or CLIP), so a query such as "slide with the architecture diagram" matches frames directly.

Results are fused with reciprocal rank fusion, then passed through **temporal fusion**: hits are grouped into their scene Segments and a scene's score aggregates its hits with a decay for distance from the densest cluster. This rewards a scene where the transcript, on-screen text, and description all agree over a scene with one lucky keyword match.

Filters: video ids, time range, source kind (transcript, ocr, description, frame), speaker, entity.

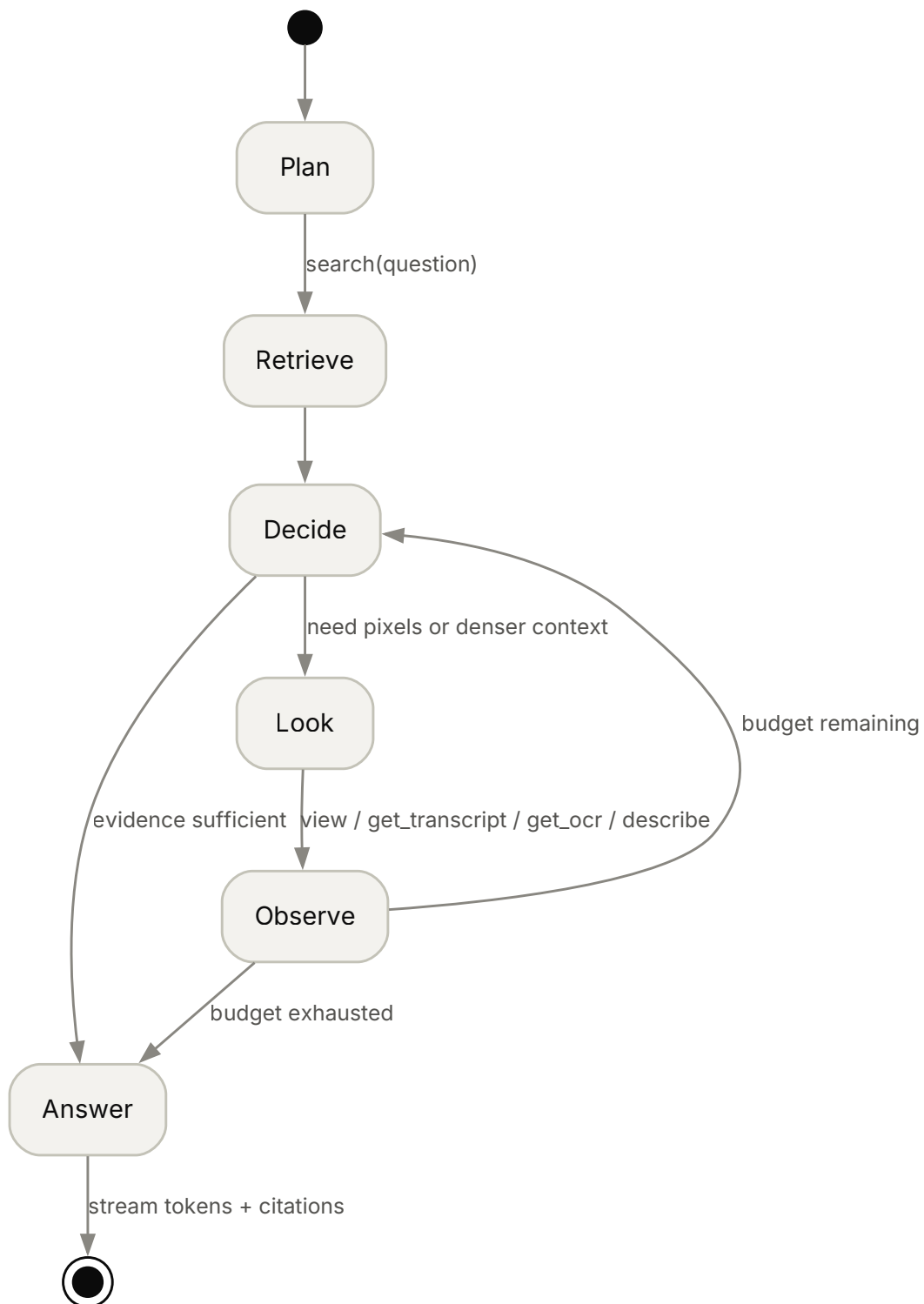
Optional **reranking** with a cross-encoder provider or an LLM over the top 30 candidates.

Output

```
{
  "hits": [
    {
      "video_id": "01J...",
      "segment_id": "01J...",
      "t0": 1834.2, "t1": 1901.7,
      "score": 0.83,
      "evidence": [
        {"kind": "transcript", "t0": 1840.0, "t1": 1852.5, "text": "...so the retriever returns ..."},
        {"kind": "ocr", "t": 1845.0, "text": "Hybrid retrieval: BM25 + dense"},
        {"kind": "description", "text": "Speaker at podium; slide showing a two-column diagram ..."}
      ],
      "thumbnail": "blob:ab/cd/..."
    }
  ],
  "index_state": "fine"
}
```

The agentic loop

The loop follows the agentic-video idea: the model chooses what to watch, at what rate, through which modality, under a budget.



The loop is implemented in `vi-agent` around a `Policy` trait:

```
#[async_trait]
pub trait Policy: Send + Sync {
    async fn next_step(&self, state: &AgentState) -> Result<Step>; // Step::Tool(call) | Step::Answer
}
```

The default policy is a tool-using LLM: it receives the question, the retrieval hits, the observations so far, and the remaining budget, and picks a tool call or answers. Python and JS can supply their own `Policy` through the bridge. Fixed strategies such as "always view the top-3 scenes at 1 fps then answer" are also policies and are useful as eval baselines.

Tools

Every tool is read-only against the index and the media. Tools are exposed identically to the internal LLM, to Python/JS callers, and over MCP.

Tool	Arguments	Returns	Cost class
<code>search</code>	query, filters, k	ranked hits with evidence	cheap, cached
<code>timeline</code>	video_id, level	segments with titles and summaries	cheap
<code>get_transcript</code>	video_id, t0, t1	transcript text with timestamps and speakers	cheap
<code>get_ocr</code>	video_id, t0, t1	on-screen text with timestamps	cheap
<code>get_descriptions</code>	video_id, t0, t1	existing VLM descriptions	cheap
<code>view</code>	video_id, t0, t1, fps, resolution, layout	frame grid image(s) with timestamp labels, plus transcript for the window	decode + VLM tokens
<code>describe</code>	video_id, t0, t1, question?	runs the VLM on the window, stores the Description, returns text	decode + VLM call, improves index
<code>listen</code>	video_id, t0, t1	audio clip for audio-capable providers, else transcript	decode, maybe provider
<code>find_similar_frames</code>	frame_sample_id or image	frames visually similar across the index	cheap

`view` and `describe` differ in who does the looking. `view` hands pixels to the loop's own multimodal LLM; `describe` calls the configured VLM provider and persists the result. Policies choose based on the loop model's capabilities and cost.

Tool arguments are validated against JSON Schema; $t_0 < t_1$, windows are clamped to configured maxima (default 120 s per `view`), and $\text{fps} \times \text{duration}$ is capped by the remaining token budget.

Budgets

An `ask` carries `Budget { max_tokens, max_cost_usd, max_wallclock, max_tool_calls }`. The loop checks the budget before each tool call and passes the remainder to the policy so it can plan. On exhaustion the loop answers with what it has and sets `partial: true` with the reason.

Coarse to fine in practice

For "When does the speaker first mention retrieval evaluation?" the default policy typically runs: `search` → answer, one LLM call, no decode. For "What color is the shirt of the person who asks the second question?" it runs: `search` for question-and-answer segments → `view` the candidate at 1 fps → answer. For "Summarize the workshop" it runs: `timeline` → `get_descriptions` per chapter → answer.

Answers and citations

Streaming events over the SDK and over SSE:

```
{
  "type": "status",
  "text": "searching"
}
{
  "type": "tool_call",
  "tool": "view",
  "args": {
    "t0": 1834,
    "t1": 1860,
    "fps": 1
  }
}
{
  "type": "tool_result",
  "tool": "view",
  "summary": "9 frames, 3 distinct"
}
{
  "type": "token",
  "text": "The speaker introduces "
}
{
  "type": "citation",
  "video_id": "01J...",
  "t0": 1840.0,
  "t1": 1852.5,
  "kind": "transcript"
}
{
  "type": "token",
  "text": "hybrid retrieval at "
}
{
  "type": "done",
  "partial": false,
  "usage": {
    "tokens_in": 8123,
    "tokens_out": 412,
    "cost_usd": 0.021,
    "tool_calls": 2,
    "wallclock_ms": 4310
  }
}
```

Citations are emitted inline as the answer streams and are always backed by a stored row (span, description, or the frames a `view` returned, which are persisted as a blob so the citation can be rendered later). The chat app renders citations as clickable timestamps that seek the player.

Implementation (M2): the model writes markers of the form `[[cite:VIDEO_ID:T0-T1]]` (seconds, or `HH:MM:SS`); a streaming scanner removes them from the token stream and emits `citation` events, typed by the evidence the loop has already seen for that video and range (`transcript` , `ocr` , `frame` , `description` , else `range`). Markers naming a video that is not in the index are dropped. The CLI prints them as `[HH:MM:SS]` (with the video title when the index holds several videos).

Multi-video and cross-video queries

An Index holds many Videos. `search` and `ask` accept a video filter; without one they run across the Index. Temporal fusion is per video; cross-video ranking uses the fused scene scores. Answers cite per video. This is how the demo app answers questions across a full playlist.

Conversation state

`ask` accepts an optional `session_id`. Sessions store prior turns and prior observations so follow-up questions reuse `view` results without re-decoding. Sessions live in the Index's SQLite under a `sessions` table with a TTL, or in memory for library use.

MCP server

`vi-server` exposes an MCP server with the tools above, plus `list_videos` and `index_state`. An external agent such as Claude Code or a LangGraph graph can therefore run its own loop over a VideoIndex, which is also how VideoIndex is compared against other agent strategies in evaluation. Resources expose thumbnails and frame grids by blob URI.

Performance targets

Operation	Target (8-core server, warm cache, no GPU)
<code>search</code> over 100 hours	< 50 ms
<code>view</code> 30 s at 1 fps, 720p	< 400 ms to grid image
<code>ask</code> , retrieval-only path	first token < 2 s plus provider latency
<code>ask</code> , one <code>view</code>	first token < 4 s plus provider latency

07. Model providers

VideoIndex never depends on a specific model. Every model capability is a Rust trait; concrete adapters implement them; configuration selects which adapter serves which role. The same Index can be built with one set of providers and queried with another, and the eval harness sweeps configurations to compare them.

Capability traits

Trait	Purpose	Key method
Vlm	Describe or answer over images, frame grids, or native video clips	<code>generate(VlmRequest) → Stream<VlmEvent></code>
Llm	Text-only reasoning, tool use for the agent loop, entity extraction	<code>generate(LlmRequest) → Stream<LlmEvent></code>
Asr	Speech to timed text	<code>transcribe(AudioChunk, AsrOptions) → Vec<TranscriptSpan></code>
Ocr	Text and boxes from a frame	<code>read(Image) → Vec<OcrSpan></code>
TextEmbedder	Text to vector	<code>embed(&[String]) → Vec<Vector></code>
ImageEmbedder	Image and text to a shared space	<code>embed_images(&[Image]), embed_text(&[String])</code>
Reranker	Score (query, passage) pairs	<code>rerank(query, &[String]) → Vec<f32></code>

One adapter may implement several traits. A Gemini adapter implements `Vlm`, `Llm`, `Asr` (via audio input), `Ocr` (via a prompt), and `TextEmbedder`.

Capabilities negotiation

Adapters report what they can do so operators and policies adapt without special-casing vendors:

```
pub struct VlmCapabilities {
    pub native_video: bool,           // accepts a video clip, not just images
    pub native_audio: bool,
    pub max_images_per_request: u32,
    pub max_image_pixels: u32,
    pub supports_tools: bool,
    pub supports_streaming: bool,
    pub supports_json_schema: bool,
    pub context_tokens: u32,
    pub price: Pricing,               // per input/output token, per image, per second of video
}
```

Examples of how this is used: - `vlm_describe` sends a native clip to a provider with `native_video`, and a 3×3 frame grid otherwise. - The agent's `view` tool sizes the grid to `max_image_pixels` and `max_images_per_request`. - The policy sees `price` so it can weigh a `view` against remaining budget.

Adapters

Adapter	Traits	Notes
<code>openai_compat</code>	Vlm, Llm, Asr, TextEmbedder	Chat completions, embeddings, audio transcription endpoints. Covers OpenAI, vLLM, Ollama, LM Studio, llama.cpp server, Together, Groq, and Whisper servers. The single most important adapter: any open model served by vLLM plugs in here.

Adapter	Traits	Notes
<code>gemini</code>	Vlm (native video, audio), Llm, Asr, TextEmbedder	Gemini API with file upload for clips longer than inline limits.
<code>anthropic</code>	Vlm (images), Llm	Claude models. Frame grids for video.
<code>onnx_local</code>	ImageEmbedder, TextEmbedder, Ocr, plus VAD internally	In-process via ONNX Runtime: SigLIP or CLIP, a small text embedder, RapidOCR, Silero VAD. CPU by default, CUDA or CoreML execution providers when available.
<code>candle_local</code>	ImageEmbedder, TextEmbedder	Alternative pure-Rust backend for platforms where ONNX Runtime is awkward.
<code>whisper_local</code>	Asr	whisper.cpp via its server or bindings, for fully offline indexing. Not built: the faster-whisper server in <code>scripts/asr-server/</code> is reached through <code>openai_compat</code> , which covers this case.
<code>paddleocr_sidecar</code>	Ocr	Optional HTTP sidecar for higher-quality OCR.

Status (2026-09-11): `openai_compat` (ASR, chat, embeddings), `anthropic`, `gemini` and `onnx_local` (SigLIP, bge-small, RapidOCR; Silero VAD is used by the `vad` operator directly) are implemented in `vi-providers` and `vi-perceive`. Gemini's file upload for clips over the inline limit, `candle_local` and `paddleocr_sidecar` are not. Adapters that live outside `vi-providers` (`onnx_local`) register through `ProviderRegistry::register_factory`.

Adding an adapter is one crate module implementing the traits and a capabilities struct. No other crate changes.

Provider configuration

Roles bind a capability to an adapter and model. The eval harness varies these.

```
[providers.gemini]
adapter = "gemini"
api_key_env = "GEMINI_API_KEY"

[providers.vllm_qwen]
adapter = "openai_compat"
base_url = "http://localhost:8000/v1"
model = "Qwen/Qwen2.5-VL-32B-Instruct"

[providers.local]
adapter = "onnx_local"
image_model = "siglip-so400m-patch14-384"
text_model = "bge-small-en-v1.5"

[roles]
vlm_describe = { provider = "gemini", model = "gemini-2.5-flash" }
agent_llm   = { provider = "anthropic", model = "claude-sonnet-5" }
asr         = { provider = "openai_compat", model = "whisper-large-v3", base_url = "http://localhost:9000/v1" }
ocr         = { provider = "local" }
text_embed  = { provider = "local" }
image_embed = { provider = "local" }
```

Roles used by the pipeline: `asr`, `ocr`, `image_embed`, `text_embed`, `vlm_describe`, `extract_llm`. Roles used by the agent: `agent_llm`, `agent_vlm` (defaults to `agent_llm` if multimodal), `reranker`.

Cross-cutting behavior in `vi-providers`

- **Rate limiting:** token bucket per provider for requests and for tokens per minute, configured or learned from 429 headers.
- **Concurrency:** semaphore per provider.

- **Retries:** exponential backoff with jitter on 429, 5xx, and transport errors; idempotent by construction since all calls are pure.
- **Batching:** embedders batch to the provider's maximum; ASR batches chunks when the endpoint allows.
- **Streaming:** SSE and chunked parsers normalized to `LLmEvent { Token, ToolCall, Usage, Done }`.
- **Cost accounting:** every call records tokens, images, seconds of media, latency, and computed cost into a `Provenance` row. Costs come from the adapter's pricing table, overridable in config.
- **Prompt registry:** prompts are versioned assets in `vi-providers/prompts/*.md` with a hash; the hash is part of every cache key and `Provenance` row. Python and JS can register overrides at runtime.
- **Structured output:** operators that need JSON request it via `supports_json_schema` when available and fall back to instruction-plus-repair otherwise.
- **Redaction:** keys never appear in logs, errors, or `Provenance`.

A/B evaluation support

Because Descriptions and Embeddings carry Provenance, several providers' outputs coexist in one Index. A query can be pinned to a Provenance filter ("only Descriptions from run X") so two configurations are compared over identical retrieval inputs. The harness in [08-evaluation](#) uses this to separate "the VLM described better" from "the agent looked in the right place".

Local versus remote decision guide

Capability	Default	Why
VAD	local	Tiny model, runs everywhere
Image embeddings	local	SigLIP on CPU does 20 to 50 images/s; avoids per-image API cost for tens of thousands of frames
Text embeddings	local	Small models are good enough for retrieval; remote is optional
OCR	local by default, VLM for hard cases	RapidOCR is fast; VLM OCR is better on stylized text
ASR	remote or local server	Whisper-large quality matters; run it under vLLM or whisper.cpp on the GPU box if one exists
VLM describe	remote	Frontier quality per dollar; open models via vLLM for A/B
Agent LLM	remote	Tool use quality dominates answer accuracy

08. Evaluation

The point of the provider abstraction is measurement. The eval harness runs the three benchmarks named in the README end to end, over the same indexes, across provider and policy configurations, and reports accuracy against cost.

Benchmarks

Benchmark	Content	Questions	Format	Source
LVBench (zai-org, ICCV 2025)	103 YouTube videos, each over 30 min, average 68 min, about 117 hours total	1,549	multiple choice; six task types: entity recognition, event understanding, key information retrieval, temporal grounding, reasoning, summarization	github.com/zai-org/LVBench; videos must be downloaded from YouTube
Minerva (Google DeepMind, 2025)	Videos averaging about 12 min	1,515 hand-crafted	multiple choice with reasoning traces; emphasizes multi-step reasoning	DeepMind release; confirm current download location during M4
1H-VideoQA (Google DeepMind)	21 YouTube videos, 40 to 90 min	101	five-way multiple choice, collected in-house	github.com/google-deepmind/1h-videoqa (Apache-2.0), data via the linked Kaggle benchmark; videos must be downloaded from YouTube. Small set, so report per-question results and a confidence interval, not just a percentage.

Google's agentic-video announcement reports up to 88% token reduction, up to 66% cost reduction, and up to 7% accuracy gain over uniform 1 fps sampling. Those three numbers, accuracy, tokens, and cost relative to a uniform-sampling baseline, are the headline metrics here too.

Harness design

The harness lives in `eval/` and is Python. It uses the SDK; the SDK has no knowledge of benchmarks.

```
eval/
  datasets/
    lvbench.py      loader: questions, options, answers, video ids, task type
    minerva.py
    onehour_videoqa.py
    acquire.py      maps benchmark video ids to Sources, drives `vi acquire`
  runners/
    index.py        builds indexes per policy config, reuses caches
    answer.py        runs `ask` per question with a fixed budget, records everything
    baselines.py     uniform-sampling baseline: N frames + transcript in one VLM call
  metrics.py        accuracy, per-task accuracy, tokens, cost, latency, tool-call histograms
  report.py          tables and pareto plots (accuracy vs cost), markdown + JSON
  configs/
    *.toml           provider roles + policy + budget matrices
```

Run structure

1. **Acquire**: download benchmark videos once. All three benchmarks are YouTube content and YouTube blocks most downloads from datacenter IPs, so downloads run on the development Mac with `scripts/download_videos.sh` and are transferred to azuremc's media cache with rsync. LVBench alone is about 117 hours, roughly 100 to 150 GB at 720p; budget disk and a day or more of residential bandwidth.

2. **Index:** for each index configuration (sampling policy × provider roles), build or extend the index. Caching means changing only the VLM re-runs only the VLM stage.
3. **Answer:** for each question, call `ask` with the benchmark's option list in the prompt and a fixed Budget. Record the streamed event log, tool calls, usage, and the final choice. The answer parser extracts the option letter; unparseable answers count as wrong and are logged.
4. **Score:** accuracy overall and per task type; mean and p95 tokens, cost, wall-clock, tool calls per question.
5. **Report:** one markdown table per benchmark per run set and a pareto plot of accuracy versus cost per question with one point per configuration.

Configurations to compare first

Axis	Values
Agent LLM	Gemini 2.5 Flash, Claude Sonnet 5, an open model via vLLM
VLM describe	same three
Policy	retrieval-only; default agentic; fixed "view top-3 scenes"; uniform-sampling baseline (no index)
Sampling	0.5, 1, 2 fps coarse pass
Budget	20k, 50k, 150k tokens per question

Controls

- Temperature 0 where the provider allows it, fixed seeds where it applies.
- Every run pins prompt hashes and provider model versions; the report lists them.
- Retrieval-only and uniform-sampling baselines run on every benchmark so the marginal value of the agentic loop and of the index are each visible.
- Contamination note: benchmark videos are public YouTube content and may be in model training data. Report it, and prefer per-configuration deltas over absolute scores.

Metrics

Metric	Definition
Accuracy	fraction of questions with the correct option
Accuracy by task type	LVBench's six categories, Minerva's categories
Tokens per question	input plus output across all provider calls in the <code>ask</code>
Cost per question	from Provenance rows, USD
Latency	wall-clock per <code>ask</code> , p50 and p95
Index cost	USD and wall-clock per hour of video, per policy
Tool-call profile	histogram of tool sequences; fraction of questions answered without decode
Citation precision	on a manually labeled subset, fraction of citations whose window contains the evidence

Development dataset

`dataset/videolist.md` lists two YouTube playlists: AI Engineer conference workshops (multi-hour sessions) and the Berkeley AI MOOC 2025. These are the demo app's content and the development smoke set. They are long, lecture-style, slide-heavy videos, which stresses OCR, transcript search, and chapter segmentation, and matches the LVBench and 1H-VideoQA regime. A small hand-written QA set of about 50 questions with timestamps over these videos serves as a fast regression test that runs on every change to prompts or policies.

Acquisition runs on the development Mac, not on azuremc, because YouTube blocks most datacenter IPs. `scripts/download_videos.sh` installs yt-dlp via Homebrew if missing, downloads the playlists at 720p with subtitles, chapters, and `.info.json` metadata into `dataset/videos/`, keeps a download archive so re-runs only fetch new entries, and prints the rsync command that moves the files into `/data/videoindex/videos/` on azuremc. The same script takes a plain-text file of URLs, which is how benchmark video lists are fetched. On azuremc the `LocalFile` acquirer imports the `.info.json` and subtitle sidecars so transferred downloads keep their metadata.

Regression gates

- The 50-question dev set runs in CI nightly against the default configuration; accuracy drops of more than 3 points fail the run.
- Unit fixtures: a 2-minute synthetic video with known cuts, burned-in text, and a scripted voice track validates shot detection, OCR, and ASR alignment deterministically without network.

09. SDK and APIs

Four surfaces, one core. The Python and Node APIs are the same shape. The CLI and server expose the same operations for scripts and for other languages.

Python

```
import videoindex as vi

idx = vi.Index.create("./talks.vidx")          # or vi.Index.open(path)
idx.configure(vi.Config.from_file("videoindex.toml"))

job = idx.add("https://www.youtube.com/watch?v=0KEGJ5G3foU&list=PLcfp...",
              policy="lecture_default",
              budget=vi.Budget(max_cost_usd=5.0))
for ev in job.progress():                     # blocking iterator; job.aprogress() is async
    print(ev.stage, f"{ev.fraction:.0%}", ev.cost_usd)
job.wait()

hits = idx.search("hybrid retrieval evaluation", k=10)
for h in hits:
    print(h.video_id, h.t0, h.t1, h.score, h.evidence[0].text)

for ev in idx.ask("When does the speaker first mention evaluating retrievers?",
                  budget=vi.Budget(max_tokens=50_000)):
    if ev.type == "token": print(ev.text, end="")
    elif ev.type == "citation": print(f" [{ev.t0:.0f}s]", end="")

answer = idx.ask("...").collect()              # non-streaming convenience
answer.text, answer.citations, answer.usage, answer.partial

# async
async for ev in idx.aask("..."):
    ...

# frames without copying: numpy array over the core buffer
frame = idx.frame(video_id, t=1834.5)         # HxWx3 uint8
grid = idx.view(video_id, t0=1830, t1=1860, fps=1).image # PIL-compatible

# extension points
@vi.operator(id="my_captioner", version=1, inputs=["scene"], outputs=["description"])
def my_captioner(ctx, scene):
    ...                                     # returns row dicts; the core stores them
idx.register_operator(my_captioner)
idx.add(url, policy={"coarse": [...], "fine": ["scenes", "my_captioner"]}) # inline policy

class MyPolicy(vi.Policy):
    def next_step(self, state): ...         # {"tool": name, "args": {...}} or None
idx.ask("...", policy=MyPolicy())

vi.prompts.override("vlm_describe", open("my_prompt.md").read())
```

Packaging: `pip install videoindex.abi3` wheels for manylinux x86_64 and aarch64, macOS arm64 and x86_64, Windows x86_64. ffmpeg libraries are statically linked into the wheel. ONNX Runtime is bundled for CPU; a `videoindex[cuda]` extra pulls the CUDA execution provider. yt-dlp is an optional runtime dependency detected on PATH.

Node.js

```
import { Index, Budget } from "@videoindex/core";

const idx = await Index.create("./talks.vidx");
const job = await idx.add(url, { policy: "lecture_default", budget: { maxCostUsd: 5 } });
for await (const ev of job.progress()) console.log(ev.stage, ev.fraction);

const hits = await idx.search("hybrid retrieval evaluation", { k: 10 });

for await (const ev of idx.ask("When does the speaker first mention evaluating retrievers?")) {
    if (ev.type === "token") process.stdout.write(ev.text);
}
```

```
const grid = await idx.view(videoId, { t0: 1830, t1: 1860, fps: 1 }); // { image: Uint8Array, width, height, timestamps }
```

Packaging: `npm install @videoindex/core` with optional platform packages holding the prebuilt `.node` binary, the napi-rs standard layout. TypeScript types shipped.

CLI

```
vi init <index-dir>                create an index
vi acquire <source>... [--out DIR]  download/copy sources into the media cache (yt-dlp for video
sites)
vi index <index-dir> <source>... [--policy P] [--budget ...]  acquire + index, streams progress
vi search <index-dir> "<query>" [--k N] [--json]
vi ask <index-dir> "<question>" [--budget ...] [--json]        streams answer with [HH:MM:SS] citations
vi view <index-dir> <video-id> --t0 .. --t1 .. --fps .. -o grid.png
vi timeline <index-dir> <video-id>
vi status <index-dir>                videos, states, sizes, costs
vi serve [--index-root DIR] [--bind 0.0.0.0:8080] [--mcp]
vi eval <config.toml>               runs the eval harness (shells to Python in eval/)
vi doctor                           checks ffmpeg, hardware decode, yt-dlp, ONNX providers, GPU
```

All commands take `--config videoindex.toml` and `--json` for machine-readable output. Exit codes are stable.

Server (HTTP)

Base path `/v1`. JSON in, JSON or SSE out. OpenAPI document served at `/v1/openapi.json`.

Method	Path	Purpose
POST	<code>/indexes</code>	create an index (hosted: per tenant)
GET	<code>/indexes/{id}</code>	status, videos, sizes
POST	<code>/indexes/{id}/videos</code>	add sources; returns job id
GET	<code>/jobs/{id}</code>	job status; Accept: text/event-stream for progress
DELETE	<code>/jobs/{id}</code>	cancel
POST	<code>/indexes/{id}/search</code>	hybrid search
POST	<code>/indexes/{id}/ask</code>	agentic answer; SSE stream of the events in 06
POST	<code>/indexes/{id}/view</code>	frame grid as PNG or WebP
GET	<code>/indexes/{id}/videos/{vid}/timeline</code>	segments
GET	<code>/indexes/{id}/videos/{vid}/transcript?t0&t1</code>	spans
GET	<code>/blobs/{key}</code>	thumbnails, grids
GET	<code>/mcp</code>	MCP endpoint (streamable HTTP)
GET	<code>/healthz</code> , <code>/metrics</code>	health, Prometheus

Auth: bearer API keys. Hosted mode adds tenants, quotas per key, and usage records. Self-hosted mode can run with auth disabled on localhost.

Configuration file

One `videoindex.toml` shared by all surfaces: providers and roles as in [07-model-providers](#), policies as in [05-indexing-pipeline](#), storage backend, media cache directory, decode worker limits, server bind and auth. Environment variables override with a `VI_` prefix.

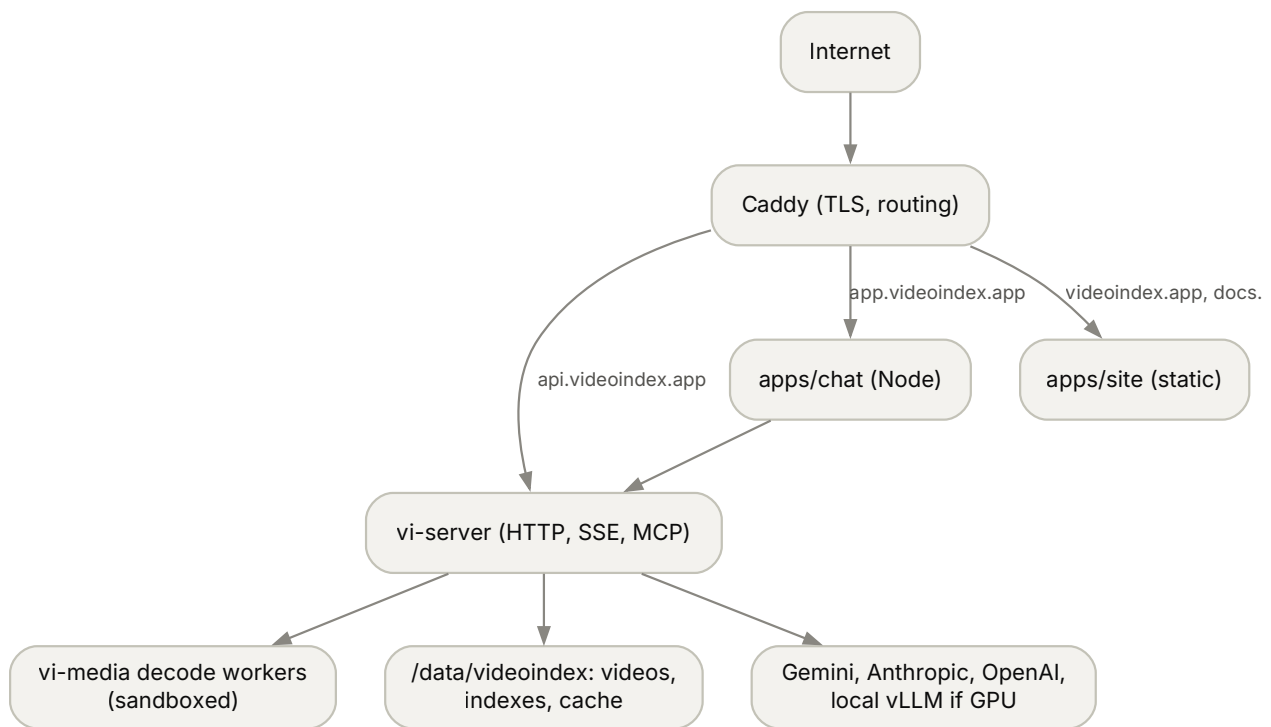
Stability policy

- Semantic versioning. Pre-1.0: minor versions may break the API with a changelog entry and a migration note.
- The index schema version is independent of the package version; readers refuse newer schemas and migrate older ones.
- Event types in `ask` streams are additive; consumers must ignore unknown types.
- The MCP tool set and the HTTP API are versioned under `/v1`.

11. Deployment: videoindex.app on azuremc

`azuremc` is a cloud machine (Azure) that hosts the `videoindex.app` domain. It runs the infrastructure, the demo QnA chat app over the dataset videos, and the SDK site. Development happens on it directly with this repository.

What runs where



Component	Host	Process	Port
Reverse proxy	azuremc	Caddy, automatic Let's Encrypt	80, 443
API + MCP	azuremc	vi serve , systemd	127.0.0.1:8080
Chat app	azuremc	Node, systemd	127.0.0.1:3000
SDK site	azuremc	static files served by Caddy	—
vLLM (optional, if GPU)	azuremc	container	127.0.0.1:8000
Whisper server (optional)	azuremc	container	127.0.0.1:9000

Caddy is chosen for automatic TLS with no cron; nginx with certbot is an acceptable substitute if it is already present on the machine.

Subdomains

- `videoindex.app` , `www.` : SDK landing and docs (later a Node.js frontend; static to start).
- `app.videoindex.app` : demo QnA chat app.
- `api.videoindex.app` : `vi-server` , including `/v1/mcp` .

Disk layout

```

/data/videoindex/
  videos/           content-addressed media cache (downloads from yt-dlp or transfers from the Mac)
  indexes/
    dataset.vidx/   the demo index over dataset/videolist.md
    eval-lvbench.vidx/
  cache/           operator cache shared across indexes (optional; else inside each index)
  logs/
/etc/videoindex/
  videoindex.toml   providers, roles, policies, storage, server
  env              API keys, mode 0600, loaded by systemd EnvironmentFile

```

Sizing: LVBench alone is about 117 hours of video, roughly 100 to 150 GB at 720p, plus 5 to 10 GB of index. The dataset playlists are tens of hours. Plan for at least 500 GB on the data volume.

systemd units

- `videoindex-api.service`: `vi serve --config /etc/videoindex/videoindex.toml --bind 127.0.0.1:8080 --mcp, Restart=always, EnvironmentFile=/etc/videoindex/env`, runs as user `videoindex`, `ProtectSystem=strict`, `ReadWritePaths=/data/videoindex`, `MemoryMax` set to leave headroom for decode workers.
- `videoindex-chat.service`: the Node chat app with `VI_API_URL=http://127.0.0.1:8080`.
- Decode workers are children of `vi serve`; the service unit's limits bound them. Seccomp for the worker is applied by `vi-media` itself.

Secrets

Provider keys live only in `/etc/videoindex/env`. The chat app holds no provider keys; it talks to `vi-server` with a server-side API key. Public demo traffic gets a rate-limited key with a per-day cost cap enforced by `vi-server` quotas so a burst of visitors cannot run up provider bills.

GPU detection

`vi doctor` and `vi serve` at startup report CPU count, RAM, GPU (via `nvidia-smi` when present), hardware decoders, ONNX execution providers, ffmpeg version, and yt-dlp presence. Configuration decides the rest:

- **GPU present**: run whisper-large under a local server for ASR, SigLIP embeddings on CUDA, optionally an open VLM under vLLM for A/B against frontier APIs.
- **No GPU**: ASR via a hosted Whisper-compatible endpoint or Gemini audio; embeddings on CPU (fine at 1 fps); all VLM and LLM calls remote.

Getting videos onto azuremc

YouTube blocks most downloads from datacenter IPs, so `vi acquire` with yt-dlp is not expected to work on azuremc. The working path is:

1. On the development Mac, run `scripts/download_videos.sh` (defaults to the playlists in `dataset/videolist.md`; pass a file of URLs for benchmark sets). It downloads 720p MP4 plus subtitles and `.info.json` into `dataset/videos/`.
2. Transfer with `rsync -avP --partial dataset/videos/ azuremc:/data/videoindex/videos/incoming/`. The script prints the exact command. Resumable, so it can run overnight.
3. On azuremc, `vi index ./dataset.vidx /data/videoindex/videos/incoming/**/*.mp4`. The `LocalFile` acquirer reads the sidecar `.info.json` and subtitle files for title, chapters, and captions, then moves the media into the content-addressed cache.

`vi acquire` on azuremc remains useful for direct HTTP and object-storage sources.

Observability

- `vi-server` exposes `/metrics` for Prometheus: request latency, tool-call counts, provider tokens and cost, decode time, cache hit rate, job states.
- Structured JSON logs to journald; keys redacted.
- Cost dashboard from the Provenance table: cost per video, per configuration, per day.

Backups

Indexes are directories. A nightly `tar` of `/data/videoindex/indexes` to Azure Blob Storage, retained 14 days, is enough at this stage. Media is re-downloadable and is not backed up.

Rollout

1. Provision `/data/videoindex`, user `videoindex`, Caddy, systemd units, DNS records for the three hostnames.
2. Build `vi` from this repo on azuremc; `vi doctor`.
3. Acquire and index the dataset playlists into `dataset.vidx`.
4. Start `vi serve`; smoke test `search` and `ask` with curl.
5. Deploy the chat app; verify citations seek the player.
6. Point `videoindex.app` at the static site.

Milestones and roadmap

Milestones are ordered so that something queryable exists early and every later milestone adds a measurable capability. Each has a definition of done that can be checked from the CLI.

M0. Skeleton and decode (first)

- Cargo workspace with all `vi-*` crates stubbed; CI on Linux and macOS with `cargo fmt`, `clippy -D warnings`, tests, `cargo-deny`.
- `vi-core`: Timestamp, ids, config loading, error type, event bus.
- `vi-media`: probe; software decode of audio to 16 kHz PCM and video at N fps with keyframe seeking; sandboxed worker process; `Arc<FrameBuffer>`.
- `vi-index` embedded backend: manifest, SQLite schema v1, blob store; `Storage` trait.
- `vi-cli`: `vi init`, `vi probe`, `vi index` running only Sample + Thumbnail + PHash, `vi status`, `vi doctor`.
- Synthetic 2-minute fixture video generated by ffmpeg in tests.

Done when: `vi index` on a 1-hour MP4 produces an index directory with thumbnails and pHashes in under 5 minutes on 8 cores, and `vi status` reports it.

Status: done 2026-09-11. Measured 2 min 48 s for a 1-hour 720p file on azuremc's 4 vCPUs (see `MACHINE-azuremc.md`). The vector store is a stub and sandboxing is rlimits only; both are M1 work.

M1. Coarse index and search

- `YtDlp`, `Http`, `ObjectStore` acquirers; playlist expansion.
- Operators: VAD, ASR (`openai_compat`), `ShotBoundary`, `ImageEmbed` (`onnx_local SigLIP`), OCR (`onnx_local RapidOCR`), subtitle import.
- Lance vectors, FTS5 text, hybrid search with RRF and temporal fusion.
- Scheduler with checkpoints, cache, budgets, progress events.
- `vi search`.

Done when: the two playlists in `dataset/videolist.md` index to `coarse` unattended, and `vi search` answers transcript, OCR, and visual queries with correct timestamps on the 50-question dev set's retrieval subset.

Progress 2026-09-11: `LocalFile` sidecar import (`.info.json`, `.srt`, `.vtt`), the content-addressed media cache move from `incoming/`, chapter import, the `subtitle_import` operator, directory expansion, and the `YtDlp` acquirer with playlist expansion are done. `vi search` exists over the text index (BM25 per kind, RRF, chapter grouping, thumbnails). Remaining: `Http` and `ObjectStore` acquirers, VAD, ASR, `ShotBoundary`, `ImageEmbed`, OCR, Lance vectors and vector search in the fusion, budgets.

Progress 2026-09-11 (GCP machine): `vi-providers` skeleton: capability traits (`Vlm`, `Llm`, `Asr`, `Ocr`, `TextEmbedder`, `ImageEmbedder`, `Reranker`) with request/response types, `ProviderRegistry` binding `[roles]` to adapters, `Governor` (per-provider semaphore, request and token buckets), retries with exponential backoff and jitter honouring `Retry-After`, `CallStats` → `Provenance`, key redaction, and the `openai_compat` adapter's ASR method (`verbose_json` with word timings, tested against a fake server). A local Whisper server runs on the GPU (`scripts/asr_server.sh`). Then the `vad` operator (Silero through ONNX Runtime, batched, `SpeechRange` items carry PCM) and the `asr` operator (concurrent requests through the `asr` role, word-level timings, per-request provenance, spans split on word timings). Measured on a 34-minute dataset video with `config/gcp-a100.toml` and the `coarse_asr` policy: 32 min of speech transcribed in 65 s (29× real time), whole job 81 s, bounded by video decode. `ShotBoundary` (HSV

histogram + edge change ratio, adaptive threshold, gap-free shot segments; 12 shots on the fixture). `ImageEmbed` (SigLIP base through ONNX Runtime, pHash-distinct frames) and `TextEmbed` (bge-small) through the `onnx_local` adapter, with a real vector store (flat memory-mapped files, exact search) behind `Storage::vector_search`. Hybrid `vi_search` (BM25 + text-vector + image-vector lists, RRF, grouping by scenes/shots/chapters/windows, `--kind frame`, `--text-only`). `Ocr` (RapidOCR, change-gated, collapsed across frames; 841 lines from a 47-minute lecture). Scheduler: budgets enforced, operator output cache with per-stage skip/replay/run planning, provider failures recorded per range with the job continuing. `Http` and `ObjectStore` acquirers (limits, private-address refusal, resume, prefix expansion) and the `auto` default policy (`coarse_only` when roles are bound). Remaining for the definition of done: the two-playlist run and `dataset/devset.jsonl`.

M2. Fine index, agent, Python

Progress 2026-09-11 (GCP machine): `vi-providers` adapters for `openai_compat` chat and embeddings, `anthropic` and `gemini` (streaming, tools, images, native video for Gemini, cost accounting, price tables, prompt registry) and `vi-agent` (tool-using loop, budgets, citations, sessions, `RetrievalOnlyPolicy`, `vi_ask`, `vi_view`, `vi_timeline`) are done. The fine-pass operators `scenes`, `chapters`, `vlm_describe` and `entities_events` exist and `lecture_default` plans end to end when the roles are bound. The Python binding exists (`bindings/python`: `Index.create/open/add/search/ask/aask/view/frame/timeline/status`, `Budget`, `Config`, prompt overrides; smoke-tested with `pytest`). Python-side operators (`@vi.operator`, `vi.Operator`, `Index.register_operator`, inline policies) and agent policies (`vi.Policy` passed to `ask`) landed on 2026-09-12 with `pytest` coverage. The QA dev set (52 questions, `dataset/devset_qa.jsonl`) scored 96.2% with citations on 2026-09-12 (`docs/M1-REPORT.md`; 86.5% before the full-text query fix), above the 80% target; the retrieval-only baseline is 73.1%. Remaining: wheels verified in CI and a measured fine-pass run over the dataset.

- Scenes and chapters; VLM describe with frame grids and native video; entities and events; text embeddings.
- `vi-providers`: `gemini`, `anthropic`, `openai_compat` adapters with capabilities, rate limits, cost accounting, prompt registry.
- `vi-agent`: default tool-using policy, all tools in O6, budgets, streaming events, sessions.
- `vi_ask`.
- Python binding with sync and async APIs, NumPy frames, operator and policy callbacks, typed stubs, wheels in CI.

Done when: `idx.ask` from Python answers the 50-question dev set at or above 80% with citations, and the retrieval-only baseline number is reported next to it.

M3. Node, server, MCP, chat app

- Node binding with prebuilt binaries.
- `vi-server`: HTTP API, SSE, blobs, API keys, MCP.
- `apps/chat`: demo QnA app over the dataset videos with a player that seeks to citations, deployed at `videoindex.app`.
- Deployment per [11-deployment](#).

Done when: a question typed in the browser at `videoindex.app` streams an answer with clickable timestamps over the AI Engineer and Berkeley playlists.

M4. Evaluation

- `eval/` harness, loaders for LVBench and Minerva, 1H-VideoQA if obtainable, baselines, reports.

- Provider and policy sweeps on azuremc; pareto plots.
- Tune sampling, grids, prompts, and policy from results; port winning policies to Rust.

Done when: a report in `docs/results/` shows accuracy, tokens, and cost per configuration on LVBench and Minerva, including uniform-sampling and retrieval-only baselines.

M5. Hosted mode

- Postgres + pgvector + object storage backend behind the `Storage` trait.
- Tenants, quotas, usage metering, key management.
- SDK site with docs at `videoindex.app`.
- Optional: Qdrant backend, hardware decode on the server GPU, whisper.cpp local ASR.

Done when: two API keys can index and query independent indexes on `videoindex.app` with usage recorded.

Later

Live streams, UniFFI for Swift and Kotlin, WASM query-only reader, multi-index federation, video-native retrieval models.

Repository layout

As given in [02-architecture](#). `apps/` and `eval/` depend on the SDK; nothing in `crates/` depends on them.

Licensing

Proposal: Apache-2.0 for `crates/`, `bindings/`, `eval/`, and `docs/`. `apps/chat` and `apps/site` may stay proprietary if the hosted product needs it; keep them in the same repo behind a clear directory boundary or split them later.

Open questions

- Confirm the license.
- Confirm the current Minerva download location. 1H-VideoQA is at `github.com/google-deepmind/1h-videoqa` (101 questions, 21 videos), resolved.
- azuremc hardware: GPU present or not decides whether ASR and image embeddings run locally on it. `vi doctor` reports this; the kickoff prompt checks it first.
- Minimum versions: Python 3.10, Node 20. Confirm.
- Hosted tenancy model: one index per tenant, or shared indexes with row-level tenant ids.
- Whether the chat app should stay in this repo or move once the SDK is published.

M1 report: what was built and measured

Written 2026-09-11 at the end of M1 (with the start of M2 folded in). The machine is described in [MACHINE.md](#); every choice made along the way is a dated bullet in [DECISIONS.md](#).

What was built

Area	Delivered	Where
Provider layer	Capability traits, role registry, per-provider governor (semaphore, request and token buckets), retries with backoff and <code>Retry-After</code> , cost accounting into <code>Provenance</code> , key redaction, prompt registry	<code>vi-providers</code>
Adapters	<code>openai_compat</code> (ASR, chat with tools/images/JSON schema, embeddings), <code>anthropic</code> (Messages, tools, images), <code>gemini</code> (function calling, inline images and video), <code>onnx_local</code> (SigLIP, bge-small, RapidOCR)	<code>vi-providers::adapters</code> , <code>vi-perceive::onnx_local</code>
Local ASR	<code>faster-whisper large-v3</code> on the A100 behind an OpenAI-compatible server, 4 GB VRAM	<code>scripts/asr-server/</code> , <code>scripts/asr_server.sh</code>
Operators	<code>vad</code> (Silero, batched), <code>asr</code> , <code>shot_boundary</code> , <code>image_embed</code> (SigLIP), <code>ocr</code> (RapidOCR, change-gated), <code>text_embed</code> (bge), <code>scenes</code> , <code>chapters</code> , <code>vlm_describe</code> , <code>entities_events</code>	<code>vi-pipeline::ops</code> , <code>vi-perceive</code>
Storage	Flat memory-mapped vector store with exact search, schema v2, sessions, entities and events, embedding lookups	<code>vi-index</code>
Search	Hybrid <code>vi_search</code> : BM25 + text-vector + image-vector lists, RRF, grouping by scenes, shots (~60 s pieces), chapters or windows	<code>vi-query</code>
Scheduler	Budgets, operator output cache with skip/replay/run planning, per-range provider failures, <code>auto</code> default policy	<code>vi-pipeline::scheduler</code>
Acquirers	<code>Http</code> (limits, private-address refusal, resume) and <code>ObjectStore</code> (S3/GCS/Azure/R2, prefix expansion)	<code>vi-media::remote</code>
Agent	<code>vi_ask</code> : tool-using loop with <code>search</code> , <code>timeline</code> , <code>get_transcript</code> , <code>get_ocr</code> , <code>get_descriptions</code> , <code>view</code> , <code>describe</code> ; budgets; inline citations; sessions; <code>RetrievalOnlyPolicy</code> baseline; <code>vi view</code> , <code>vi timeline</code>	<code>vi-agent</code> , <code>vi-cli</code>
Python	<code>videoindex</code> package: <code>Index.create/open/add/search/ask/aask/view/frame/timeline/status</code> , <code>Budget</code> , <code>Config</code> , prompt overrides	<code>bindings/python</code>
Dev sets	72 retrieval questions with caption/OCR anchors; 52 QA questions with accepted answers	<code>dataset/devset.jsonl</code> , <code>dataset/devset_qa.jsonl</code> , <code>scripts/devset_*_eval.py</code>

Timings on this machine (12 vCPUs, one A100 80 GB)

Measurement	Result
M0 policy (sample, pHash, thumbnails), synthetic 1-hour 720p	48.6 s wall, 6.6 CPUs busy (azuremc: 167.6 s)
Coarse pass with ASR (<code>coarse_asr</code>), 34-minute talk, machine otherwise idle	81 to 91 s wall; decode-bound, ASR and embeddings in its shadow
Whisper large-v3 through the pipeline	32 minutes of speech in 65 s = 29× real time at 4 concurrent requests; a lone 60 s clip 41×
Silero VAD, 34-minute track	14 s including the audio decode, batched 32 parts in lockstep

Measurement	Result
Coarse pass without ASR (<code>visual</code>), 47-minute slide lecture, idle machine	CPU build 112 s; CUDA build 50 s after the cuDNN fix (177 s before it); decode alone 39.5 s. 247 frames read by OCR (824 to 843 lines), 168 SigLIP embeddings. Details in <code>MACHINE.md</code> .
Full dataset (36.6 h, 30 videos, <code>coarse_only</code>), one process, CPU ONNX build, builds and tests competing for the CPU	17,856 s = 4 h 58 min, 7× real time overall (5× on OCR-heavy slide decks, 23× on camera-heavy talks); 0 failures; index 1.06 GiB (187 MiB SQLite, 567 MiB thumbnails in 130,907 blobs). Per-video table below.
Fine pass (<code>full</code>) on the 34-minute talk with Claude Sonnet 5	see below
Hybrid <code>vi</code> search , one-shot CLI	1.5 to 2.4 s, of which about 1.3 s loads bge and the SigLIP text tower; the search itself under 30 ms
<code>vi</code> ask on the 34-minute talk	3 tool calls, correct timestamps, \$0.065, 14.6 s

GPU utilisation during the dataset run: the A100 was used only by Whisper (about 4.5 GB, 60 to 100% while ASR requests run). ONNX Runtime ran on the CPU because the run started before the CUDA toolkit was installed; the CUDA build (`--features cuda`) landed the same night and, after fixing cuDNN's exhaustive per-shape algorithm search, cuts the visual pass from 112 s to 50 s on the test lecture. Re-indexing the dataset on the GPU build is the obvious next timing run (expected: bounded by decode and ASR, roughly 2 h for the 36.6 h).

Per-video timings, dataset run (CPU ONNX build, loaded machine)

Video	Duration	Wall	× real time	asr	ocr	image_embed	shot_boundary	text_embed
Building Multimodal AI Agents From Scratch — Apoorva Joshi,	0:36:58	229 s	10×	207	859	133	45	1227
Evals 101 — Doug Guthrie, Braintrust	0:48:31	301 s	10×	258	1453	183	51	1920
A2A & MCP Workshop: Automating Business Processes with LLMs	1:23:13	678 s	7×	417	1451	801	61	2236
RFT, DPO, SFT: Fine-tuning with OpenAI — Ilan Bigio, OpenAI	1:46:14	2130 s	3×	652	35098	368	77	36160
Agentic AI MOOC / UC Berkeley CS294-196 Fall 2025 / Agentic	1:48:50	398 s	16×	583	3610	77	39	5225
Agentic AI MOOC / UC Berkeley CS294-196 Fall 2025 / Multi-Ag	0:54:49	159 s	21×	320	675	42	18	1484
How to build world-class AI products — Sarah Sachs (AI lead)	1:43:45	946 s	7×	556	10200	553	66	11214
Agentic AI MOOC / UC Berkeley CS294-196 Fall 2025 / Evolutio	1:19:31	209 s	23×	428	1021	80	36	2147
Collaborating with Agents in your Software Dev Workflow - Jo	1:04:06	752 s	5×	368	6221	486	44	6854
Strategies for LLM Evals (GuideLLM, lm-eval-harness, OpenAI	0:32:28	282 s	7×	177	1420	149	37	1740

Video	Duration	Wall	x real time	asr	ocr	image_embed	shot_boundary	text_embed
Agentic AI MOOC / UC Berkeley CS294-196 Fall 2025 / Post-Tra	1:17:28	225 s	21×	414	1117	13	14	2219
[Full Workshop] Reinforcement Learning, Kernels, Reasoning,	2:42:27	1490 s	7×	863	22663	220	65	24225
Shipping AI That Works: An Evaluation Framework for PMs – Am	1:26:16	862 s	6×	457	6275	733	76	7109
Agentic AI MOOC / UC Berkeley CS294-196 Fall 2025 / Practica	0:46:54	172 s	16×	252	841	168	41	1509
Agentic AI MOOC / UC Berkeley CS294-196 Fall 2025 / Predicta	0:44:04	148 s	18×	258	1208	37	17	1875
VoiceVision RAG - Integrating Visual Document Intelligence w	1:23:51	740 s	7×	503	10024	250	65	10898
The AI Engineer's Guide to Raising VC — Dani Grant (Jam), Ch	0:34:16	191 s	11×	197	580	47	6	922
Introduction to LLM serving with SGLang - Philip Kiely and Y	0:43:42	524 s	5×	254	6882	114	35	7330
Agentic AI MOOC / UC Berkeley CS294-196 Fall 2025 / Autonomo	1:01:42	590 s	6×	326	3443	469	103	4319
Agentic AI MOOC / UC Berkeley CS294-196 Fall 2025 / Training	1:04:37	275 s	14×	366	742	121	23	1673
Agentic AI MOOC / UC Berkeley CS294-196 Fall 2025 / LLM Agen	1:58:21	400 s	18×	623	2148	154	87	3839
ComfyUI Full Workshop — first workshop from ComfyAnonymous h	0:51:25	499 s	6×	297	1281	595	31	1810
[Full Workshop] Building Conversational AI Agents - Thor Sch	1:01:42	391 s	9×	347	3349	245	41	3971
How LLMs work for Web Devs: GPT in 600 lines of Vanilla JS -	1:41:33	978 s	6×	567	8070	415	83	9065
Agentic AI MOOC / UC Berkeley CS294-196 F25 / Multi-Agent Sy	0:58:58	224 s	16×	309	883	166	50	1719
Agentic AI MOOC / UC Berkeley CS294-196 Fall 2025 / AI Agent	1:01:15	206 s	18×	327	1401	79	26	2272
From Mixture of Experts to Mixture of Agents with Super Fast	0:53:15	431 s	7×	301	4421	119	38	4955

Video	Duration	Wall	x real time	asr	ocr	image_embed	shot_boundary	text_embed
Full Workshop: Realtime Voice AI — Mark Backman, Daily	1:09:40	722 s	6×	38 3	596 3	559	47	6641
[Full Workshop] Vibe Coding at Scale: Customizing AI Assista	1:20:38	1255 s	4×	541	238 00	370	66	24694
Building Code First AI Agents with Azure AI Agent Service —	1:54:05	1447 s	5×	76 2	1103 4	759	106	12289
30 videos	36:34:35	17856 s	7×					

Columns after the speed: transcript spans written by `asr`, OCR lines, SigLIP frame embeddings (pHash-distinct frames), shots, and bge text embeddings (transcript + OCR spans). Generated by `scripts/index_log_timings.py`.

Hosted-provider cost: the only paid calls were Anthropic (`claude-sonnet-5`) for the agent and the fine pass; figures are in the sections below and in each run's report line.

Dev set results

Two dev sets over the 30 dataset videos (`dataset/`): 72 retrieval questions (65 transcript, 3 OCR, 4 visual) scored by `scripts/devset_eval.py` (`vi search`, `k=5`, a hit is a result within 30 s of the anchored ground truth), and 52 QA questions with accepted answer strings scored by `scripts/devset_qa_eval.py` (`vi ask --json`, budget \$0.30 and 6 tool calls per question, `claude-sonnet-5` as `agent_llm`).

Retrieval (`vi search`, hybrid: BM25 per kind + bge + SigLIP, RRF)

	n	video@1	hit@1	hit@5	MRR
First run (FTS ANDs every word)	64 scored, 8 anchors unresolved	0.578	0.344	0.766	0.502
After OR-ing terms with stopword removal	72	0.819	0.583	0.944	0.724
transcript questions	65	0.831	0.569	0.954	0.721
OCR questions	3	1.0	1.0	1.0	1.0
visual questions (SigLIP text-to-frame)	4	0.5	0.5	0.75	0.562

The first run's failure mode was structural, not a ranking problem: the FTS5 query required every word of the question, so BM25 returned nothing for almost every natural-language query and the fusion ran on the two vector lists alone. With OR-ed terms all four lists contribute to every hit (`lists` in the eval output). The four remaining misses are two right-video-wrong-minute cases (the question's wording matches a later restatement), one visual question whose frame ranks fourth, and one paraphrase ("collision rate" is spoken as "collisions") that neither BM25 nor bge bridges. Median `vi search` latency is 2.4 s as a one-shot CLI, of which about 2 s is process start and model load (SigLIP text tower and bge on CUDA); the search itself is tens of milliseconds and the Python binding pays the load once.

Question answering (`vi ask`)

Run	accuracy	cited	citation video correct	citation within the anchor	cost / question	tool calls / question	median latency
Agent, first retrieval (AND-ed FTS)	45/52 = 0.865	0.94 2	0.942	0.712	\$0.060 (\$3.10 total)	2.46	7.1 s

Run	accuracy	cited	citation video correct	citation within the anchor	cost / question	tool calls / question	median latency
Agent, OR-ed FTS + empty-answer retry	50/52 = 0.962	1.00 0	1.000	0.808	\$0.052 (\$2.71 total)	2.04	6.6 s
Retrieval-only baseline (one search, then answer)	38/52 = 0.731	0.75 0	0.750	0.635	\$0.017 (\$0.90 total)	1.00	4.6 s

The M2 target ("idx.ask answers the dev set at or above 80% with citations, retrieval-only baseline reported next to it") is met: 96.2% with the fixed retrieval against a 73.1% retrieval-only baseline (one search, then answer, no agent loop), at a third of the cost per question. The two remaining agent misses are real: it names Notion's "AI Writer" where the speaker's answer is meeting notes, and "covariance" where the lecture's metric is the collision rate (the same speech-to-text paraphrase gap that costs retrieval question q32). The baseline's 14 misses are mostly questions whose answer sits in a span the first search does not surface, which is what the agent's extra 1.04 tool calls per question buy. Three of the seven first-run misses were empty answers: the agent spent its six tool calls searching (the AND-ed FTS starved it of candidates), and the final no-tools turn came back with no text. The loop now asks once more, explicitly, before returning a partial answer. The other four misses are genuine: an answer that names the wrong product, one that stops short of the number asked for, and two where the accepted strings are stricter than the (arguably correct) paraphrase the model gave.

What the design got wrong or left unclear

- Slide changes are not shots.** The design lists `ShotBoundary` as the visual unit and `ocr` as running "on distinct frames whose text-likeness heuristic fires". On slide lectures a title change moves as many pixels as a sponsor-logo swap; no histogram or edge threshold separates them, and pHash of a white slide ignores its text. OCR now runs on a pixel-change gate and collapses repeated lines; slide changes surface as OCR spans, and chapters use OCR title changes. Shots stay true camera cuts.
- Lance.** `04-data-model.md` names Lance as the default vector store. Its current release pulls 689 crates (Arrow, DataFusion) into a workspace of about 250, for exact search over a few hundred thousand vectors that a flat memory-mapped file does in milliseconds. The flat store ships; Lance or usearch remain a drop-in behind the trait.
- Text embeddings belong to the coarse pass.** The design put `text_embed` in the fine pass, but hybrid search needs span vectors before any VLM runs. It moved.
- Resume needed a replay hook.** "Re-running with a different VLM re-runs only the VLM stage" is only true if cached producers can hand their stored outputs to a re-running consumer. Operators got `replay()`; operators whose outputs are pixels (`sample`) cannot replay, so anything needing frames re-decodes.
- One 30 s ASR chunk per request wastes the GPU.** The design's "30 s chunks with 1 s overlap" gives 17× real time; 120 s VAD segments that the batched server cuts itself give 29×.
- Scenes need to split long shots.** A static camera holds one shot for half an hour; the design's "scenes 20 s to 3 min" only holds if long shots are cut before grouping.
- Tool results cannot carry images on OpenAI-shaped APIs,** so `view` returns text and the grid follows as a user message; Anthropic and Gemini accept this too.
- Claude 5 rejects `temperature`.** The eval doc's "temperature 0 where the provider allows it" needs the adapter to omit the field for these models.
- Sort comparators with tolerances panic.** Rust's sort checks for total orders; the OCR reading-order sort tripped it on the first long video. Every float sort uses `total_cmp` now.

10. **The kickoff assumed a 40 GB A100.** This machine has 80 GB, so a 32B VLM fits in fp16 for the M4 A/B runs without quantisation.

Open items carried into M2/M4

- ~~Install a CUDA toolkit~~ Done 2026-09-11 after asking: CUDA 13.1 + cuDNN 9, `vi` built with `--features cuda`. The models run 10 to 35× faster per call on the GPU; the coarse pass is then bound by CPU video decode (no NVDEC: no `libnvcuvid`, and the corpus is AV1/VP9). See `MACHINE.md`.
- ~~Python-side operators and policies~~ Done 2026-09-12 (`@vi.operator`, `vi.Policy`, inline policies). Still open: wheels verified in CI; the zero-copy frame view (frames are copied into NumPy today).
- `Diarize`, `listen`, `find_similar_frames`.
- Format the local SSD if build times or temp files ever matter.

Work performed

This chapter is generated from the repository at build time (2026-09-12). The project has 51 commits between 2026-09-10 and 2026-09-12; the Rust workspace holds 29,740 lines of library and binary source plus 2,785 lines of tests, examples and Python, with 154 automated tests.

Commits by type

Type	Commits	Meaning
feat	22	Features
docs	12	Documentation
fix	6	Fixes
other	4	
chore	3	Chores
ci	2	CI
perf	2	Performance

Code by crate

Crate	Source lines	Test / example / Python lines	Files	Tests
vi-agent	1,756	345	7	5
vi-cli	1,580	221	13	4
vi-core	2,314	0	8	20
vi-index	3,790	0	8	12
vi-media	4,792	514	17	33
vi-perceive	3,311	126	13	20
vi-pipeline	5,916	840	21	24
vi-providers	4,284	63	16	25
vi-query	482	225	3	2
vi-server	8	0	1	0
vi-testkit	153	0	2	2
bindings/python	1,354	451	5	7
total	29,740	2,785		154

Commit log by day

Conventional-commit subjects, oldest first. Scopes name the crates a change touched.

2026-09-10

- **other**: first commit (797f626)
- **other**: updated plan for implementing videoindex (7400565)

2026-09-11

- **docs**: record azuremc machine facts and initial decisions (158e188)

- **feat** `vi-core` : timestamps, ids, config, errors, event bus, data model (7449620)
- **chore** `vi-core` : silence clippy on figment jail test (c010657)
- **feat** `vi-media` : libav probe/decode in a sandboxed worker process (4e75edf)
- **feat** `vi-perceive` : perceptual hash and WebP thumbnails (c37c4ee)
- **feat** `vi-index` : Storage trait and embedded SQLite/FTS5/blob backend (fb26080)
- **feat** `vi-pipeline` : operator DAG, tokio scheduler, checkpoints, Sample/PHash/Thumbnail (c170e5a)
- **feat** `vi-cli` : vi init, probe, index, status, doctor (ef2e6a7)
- **chore** : cargo-deny passes; optimise deps in dev profile; fix testkit item order (a3cfb7a)
- **ci** : GitHub Actions on ubuntu and macos (fmt, clippy -D warnings, tests, doc, cargo-deny) (340a17f)
- **docs** : record M0 timing on azuremc and mark M0 done (95d2720)
- **feat** `vi-media,vi-pipeline` : sidecar import, media cache, subtitle_import, YtDlp acquirer (ab6c3e5)
- **feat** `vi-query,vi-cli` : text search with RRF and chapter grouping; vi search (c90bc7b)
- **docs** : machine recommendations for indexing/dev and serving hosts (4aa2889)
- **docs** : kickoff prompt for the GCP A100 machine; azuremc machine file renamed (baf4dc8)
- **other** : update vibe summaries (65354f5)
- **other** : downloads vibe summary (301140d)
- **docs** `machine` : GCP A100 host facts, M0 re-timing (48.6 s/hour), local Whisper server (7f80dda)
- **feat** `vi-providers` : capability traits, role registry, governor, retries, cost accounting, openai_compat ASR (3d934c4)
- **feat** `vi-perceive,vi-pipeline` : Silero VAD and Whisper ASR operators through the provider layer (c980840)
- **feat** `vi-perceive,vi-pipeline` : shot_boundary operator with gap-free shot segments (757f50b)
- **feat** `vi-index,vi-perceive,vi-pipeline` : SigLIP/bge embeddings and a flat vector store (731328d)
- **feat** `vi-query,vi-cli` : hybrid search with text and image vector lists, temporal units from shots (63ed64e)
- **feat** `vi-perceive,vi-pipeline` : RapidOCR on-screen text with change-gated reads (2e988b8)
- **feat** `vi-pipeline` : budgets, operator output cache with skip/replay/run planning, per-range provider failures (b886300)
- **feat** `vi-media,vi-core` : Http and ObjectStore acquirers; auto default policy (4bee414)
- **fix** `ocr` : total-order reading sort; total_cmp everywhere; dev set and eval script (68c8c6a)
- **feat** `vi-providers` : openai_compat chat and embeddings, anthropic and gemini adapters, SSE parser, price tables (8bbafdb)
- **feat** `vi-agent,vi-cli` : agentic ask loop with tools, budgets, citations, sessions; vi ask, view, timeline (4c6af4d)
- **feat** `vi-pipeline` : fine pass operators scenes, chapters, vlm_describe, entities_events (0ae338d)
- **feat** `bindings/python` : PyO3 binding; fix scenes for long shots and extraction JSON (f501296)
- **docs** : M1 report draft; QA dev set (52 questions) and its evaluation script (8b0bbc0)
- **fix** `vi-media` : spawn the decode worker from /proc/self/exe; relax the default wall-clock budget (12c92ad)
- **ci** : build and test the Python wheel on ubuntu and macos (e366785)
- **fix** `chapters` : titles need three words, no URL/e-mail, and an upper-frame box (8b0edff)
- **docs** : citation markers, adapter status, Python usage in the README (e3009a7)
- **fix** `vi-perceive` : share loaded ONNX models across provider handles; document CUDA 13 install (d4c0da4)
- **perf** `vi-perceive` : onnx_bench example; record GPU per-call timings and the no-NVDEC finding (54c38d0)

2026-09-12

- **feat** `python` : operators and agent policies in Python; inline policies; frame descriptions searchable (68d4e01)
- **chore** `scripts` : per-video timing table from vi index logs (ebc7809)
- **fix** `vi-pipeline` : list registered custom operators in the unknown-operator error (37be6e4)
- **docs** : M1 report open items updated for CUDA and Python operators (3c1da8f)
- **fix** `vi-index` : OR full-text terms with stopword removal; eval anchors match across caption cues (e1fb2bf)
- **perf** `vi-perceive` : heuristic cuDNN conv search and bucketed OCR widths; agent retries an empty final answer (99cabb2)
- **docs** `M1 report` : dataset timings, per-video table, retrieval and first QA results (0caad38)
- **docs** : link the M1 report; roadmap M2 progress with the QA result (e85f263)
- **docs** `M1 report` : QA rerun 96.2% and retrieval-only baseline 73.1%; vi ask accepts retrieval_only (c2a3cb6)
- **docs** `M1 report` : describe the two remaining QA misses accurately (42d0a01)
- **feat** `scripts/report` : Markdown-to-PDF technical report generator with Graphviz diagrams and charts; benchmark and team report builders; Gemini agentic-video eval harness (95ca112)

Decisions

One dated bullet per decision made where the design documents were silent or had to change. Newest at the bottom.

- 2026-09-11: `/data` is a plain directory on the root filesystem (164 G free), not a separate volume. Neither existing disk has the 500 G the deployment doc plans for; a dedicated data disk is deferred until benchmark media (M4) needs it. The ephemeral `/mnt` disk is never used for media or indexes.
- 2026-09-11: The M0 "under 5 minutes for a 1-hour MP4" target is measured on this machine's 4 vCPUs, not the 8 cores the docs assume. The number is recorded either way; if it misses on 4 cores the doc target is annotated rather than the machine upgraded.
- 2026-09-11: The M0 timing test uses a synthetic 1-hour 720p H.264 file (`ffmpeg testsrc2`, 30 fps, GOP 150) because no real hour-long video exists on the machine yet and YouTube is blocked. Real dataset videos re-run the measurement once transferred.
- 2026-09-11: `yt-dlp` is installed as the upstream standalone binary in `/usr/local/bin`, not the Ubuntu package (which is a year old and breaks against YouTube). It is only for `vi doctor` and the `YtDlp` acquirer on machines where YouTube is reachable.
- 2026-09-11: `nginx` already runs on this machine for other sites and port 8080 is taken by another app. The deployment doc's Caddy-on-80/443 and `vi serve` on 8080 will be adjusted at M3 (`nginx` site + a free localhost port); `11-deployment.md` is updated when that work happens, not now.
- 2026-09-11: `libav` is reached through `ffmpeg-next 9.0`, not `rsmpeg`. `ffmpeg-next` wraps contexts, frames, packets, scalers and resamplers in safe types, so `vi-media` holds almost no `unsafe` (only `mmap` and the pre-exec `rlimit` hook), and its build script detects any `libav` from 3.0 to 9.0 via `pkg-config`, which covers the distro `ffmpeg 6.1` here and Homebrew's current `ffmpeg` on macOS CI without pinning. `rsmpeg` pins to one `ffmpeg` release and exposes a near-raw C API.
- 2026-09-11: One decode worker process per request (probe, video decode, audio decode). Spawning costs a few milliseconds, each request is crash-isolated, and the client API stays trivial. A worker pool can be added behind the same `vi-media::client` functions when concurrency needs it.
- 2026-09-11: Frames cross the worker boundary through file-backed shared memory (`/dev/shm` on Linux, temp dir elsewhere) divided into fixed-size slots. Slot ownership moves with protocol messages: the worker owns a slot until it announces `Frame{slot}`, the parent until the `Arc<FrameBuffer>` drops and sends `SlotFree`. The slot count (`media.worker.max_in_flight_frames`, default 16) is the per-job frame bound; pipeline channel capacities are derived from it so backpressure cannot deadlock.
- 2026-09-11: Frames are scaled inside the worker to `media.sample_max_dim` (default 640 px longest side) before crossing to the parent. Nothing in the coarse pass needs more (pHash 32x32, thumbnails 320 px, SigLIP 384 px), and it cuts shared-memory traffic and `swscale` work roughly fourfold for 720p.
- 2026-09-11: When the sampling rate is far below the source frame rate the decoder skips non-reference frames (`AVDISCARD_NONREF`). That is lossless for the frames kept and cut the 1-hour 720p decode from 259 s to 139 s on this CPU (`ffmpeg` CLI baseline; the worker measured 150 s). The sampled frame can land up to two frames after the target; its true PTS is stored. Per-target keyframe seeking is used only when the sampling interval is at least twice the keyframe interval. `skip_loop_filter` (a further ~15%) is left off to keep thumbnails clean.
- 2026-09-11: Sandboxing in M0 is `rlimits` (address space, open files, no core dumps) plus `PR_SET_NO_NEW_PRIVS` on Linux, applied in a `pre_exec` hook. `seccomp` and macOS `sandbox-exec` are deferred; the process boundary and idle timeout already exist. `vi doctor` prints what is active.
- 2026-09-11: The `Storage` trait gained methods the pipeline and CLI need beyond `04-data-model.md`: `get_video`, `find_video_by_hash`, `list_videos`, `set_index_state`, `put_tracks / tracks`,

- `put_frame_samples`, `update_frame_phash`, `update_frame_thumbnail`, `delete_frame_samples`, `frame_samples`, `put_embeddings`, `list_jobs`, `stats`. The doc is updated in the same commit.
- 2026-09-11: Operators stream their outputs through `OpContext::emit` instead of returning them in `OpOutput`, and have a `finish` hook for flushing buffered writes. A `Vec` return would hold an hour of frames in memory and defeat the bounded-channel backpressure.
 - 2026-09-11: The vector store is a stub: `put_embeddings` records metadata rows in SQLite, vectors are dropped, `vector_search` returns nothing. Lance lands in M1 behind the same trait.
 - 2026-09-11: Video rows are keyed by media content hash: re-indexing the same bytes reuses the `VideoId`. Parent tables are upserted (`ON CONFLICT DO UPDATE`), never `INSERT OR REPLACE`, because `REPLACE` deletes the row and the `ON DELETE CASCADE` would wipe tracks and samples.
 - 2026-09-11: A built-in `m0` policy (`sample`, `phash`, `thumbnail`) is the default policy until M1 operators exist. Policies naming unimplemented operators (`vad`, `asr`, ...) fail with "not implemented yet" and exit code 6, distinct from typos (exit 2). After the `m0` policy the video is marked `coarse`: the state records that the policy's coarse pass finished, not that every coarse operator in the design ran.
 - 2026-09-11: `vi-testkit` is an extra workspace crate (not in the `02-architecture.md` layout) whose build script generates the 2-minute fixture with the `ffmpeg` CLI. It is a dev-dependency only, so building `vi` needs no `ffmpeg` binary. Solid-colour segments alone defeat pHash (constant luma has no DCT structure), so each segment carries a white box at a segment-specific position.
 - 2026-09-11: Lossy WebP thumbnails use the `webp` crate (bundled `libwebp`) at quality 75 because the `image` crate only writes lossless WebP, which would be roughly five times larger per hour of video.
 - 2026-09-11: Default server bind is `127.0.0.1:8090`, not 8080, because 8080 is taken on this machine (see `MACHINE-azuremc.md`). `11-deployment.md` is updated at M3 when the server exists.
 - 2026-09-11: Dev-profile dependencies are compiled with `opt-level = 2` so `libwebp` and `image` resizing run at a usable speed in tests; workspace crates stay unoptimised for debugging.
 - 2026-09-11: M1 starts with sidecar import. `LocalFile` reads `<stem>.info.json` and `<stem>[.lang].{srt,vtt}`; files under `<media.cache_dir>/incoming/` are moved (with sidecars) to `<cache_dir>/<blake3>.<ext>`, files elsewhere stay put. `source_uri` becomes the sidecar's `webpage_url` when present so a transferred download keeps its origin; the local path is in the stored probe. A duplicate in `incoming/` whose hash is already cached is deleted rather than kept.
 - 2026-09-11: Subtitle cues are grouped into spans of about 15 s (new span after a gap over 2 s) before storage; single cues are too short to rank well in BM25. YouTube's rolling two-line auto-captions are collapsed to their new material. Human-authored subtitles (listed under `subtitles` in `.info.json`) get confidence 1.0, automatic captions 0.6, so ASR (when it exists) can prefer to replace only the automatic ones. Sidecar subtitle tracks use `stream_index ≥ 1000` because they have no container stream.
 - 2026-09-11: Chapters from `.info.json` (else container chapters) are written as `chapter`. Segments during acquisition by the scheduler, not by an operator: they are metadata that needs no decoding, and the fine-pass `chapters` operator can later replace them with topic segmentation when neither source has them.
 - 2026-09-11: The default policy is now `coarse_local` (`subtitle_import`, `sample`, `phash`, `thumbnail`): everything that runs without a model provider. `m0` remains for the timing benchmark.
 - 2026-09-11: `Ytdlp` downloads into `<cache_dir>/incoming/` with the same fixed argument set as `scripts/download_videos.sh` (720p MP4, `.info.json`, subtitles and auto-captions converted to SRT, embedded chapters, download archive) and then reuses `LocalFile`, so both paths produce identical index contents. The URL is passed after `--` so user input can never be read as a flag. Playlists expand via `--flat-playlist`.
 - 2026-09-11: `vi search` (M1, text only so far) fuses per-kind BM25 lists with reciprocal rank fusion ($k = 60$), then groups evidence by the chapter segment containing it, or by 60 s windows when a video has no chapters. A group scores $\text{best} + 0.3 \times \text{sum}(\text{others})$, so a chapter where transcript and on-screen text agree outranks a single lucky keyword. Scene segments replace chapters as the grouping unit

once shot and scene detection exist. Query words are ANDed with a prefix match on the last word; quoting is escaped so user text cannot inject FTS5 syntax.

- 2026-09-11 (GCP): Development moved to a GCP `a2-ultragpu-1g` (12 vCPUs, 167 GiB, one A100 **80 GB**, Ubuntu 26.04, ffmpeg 8.0). `docs/MACHINE.md` describes it; the `azuremc` file keeps its name. The kickoff prompt assumed `a2-highgpu` with 40 GB; with 80 GB a 32B VLM fits in fp16 on one GPU, so no quantisation or tensor parallelism is planned.
- 2026-09-11 (GCP): The blank 375 GB local SSD (`/dev/nvme0n1`) was left unformatted: the kickoff session's permission policy refused the `mkfs`, and nothing needs it yet (`/dev/shm` is 84 GB, the OS disk has 715 GB free for `target/`). Format and mount it at `/mnt/localssd` by hand if build times or temp files ever matter.
- 2026-09-11 (GCP): No Docker on this machine yet. The Whisper server runs as a plain user process: `faster-whisper` in a uv-managed Python 3.12 venv (`/data/videoindex/asr/.venv` ; the system Python 3.14 has no ML wheels) with the CUDA 12 runtime supplied by the `nvidia-cublas-cu12` and `nvidia-cudnn-cu12` pip wheels, so no system CUDA install was needed. `scripts/asr-server/server.py` is a small FastAPI app implementing the OpenAI `/v1/audio/transcriptions` shape (verbose_json with word timestamps) over `BatchedInferencePipeline`; `scripts/asr_server.sh start|stop|status` manages it on `127.0.0.1:9000`. A container and a systemd unit can replace this later without touching the `openai_compat` adapter.
- 2026-09-11 (GCP): ONNX Runtime models run on the CPU until a CUDA toolkit (cudart, cuBLAS, cuDNN 9) is installed system-wide, which the kickoff says to ask about first. `vi doctor` reports whether those libraries are present. Model files live under `/data/videoindex/models/<model>/`; the operators take their paths from config and refuse to start with a download hint when a file is missing. SigLIP is `Xenova/siglip-base-patch16-224` (768-dim) because no ONNX export of `so400m-patch14-384` exists on the Hub; swapping models is a config change.
- 2026-09-11 (GCP): `vi-providers` holds the capability traits, the role registry, the per-provider `Governor` (semaphore + token buckets), retries and cost accounting, and the HTTP adapters. Local ONNX adapters implement the same traits from `vi-perceive` (which owns the ONNX Runtime sessions) and plug into the registry through `register_factory("onnx_local", ...)`, so `vi-providers` never depends on ONNX Runtime and `vi-perceive` stays the only crate that does. The pipeline registers the factory at start-up.
- 2026-09-11 (GCP): Provider tables gained typed fields (`concurrency`, `requests_per_minute`, `tokens_per_minute`, `timeout_secs`, `max_retries`, `pricing`, `model_dir`) next to the free-form `extra` map, and `vi_core::Error::Provider` (exit code 8) distinguishes a failed or unbound provider from an unsupported feature. No providers or roles are configured by default; `coarse_local` stays the default policy and provider-backed operators fail at plan time with the role that is missing.
- 2026-09-11 (GCP): Silero VAD runs through the `ort` crate (ONNX Runtime 1.23 prebuilt binaries downloaded at build time, rustls for the download, no `tracing` feature so ONNX Runtime's own INFO chatter stays out of the logs). Model files live under `models.dir` (`/data/videoindex/models` on servers, `~/.cache/videoindex/models` otherwise), one subdirectory per model; a missing file fails at operator start with the download hint. The whole audio track is decoded to memory (16 kHz mono i16, 115 MB per hour) and scored in one pass with 32 parts of the recording batched through the model in lockstep, because one ONNX Runtime call per 32 ms window cost more than the arithmetic: 24 s → 14 s for a 34-minute track including the decode.
- 2026-09-11 (GCP): VAD segments are long on purpose (`max_segment_secs` 120, `min_silence_ms` 1000): the batched Whisper server splits each request into 30 s pieces at pauses and fills its GPU batch, so 70 requests of ~27 s transcribe 32 minutes of speech in 65 s (29× real time, 4 concurrent requests), against 17× with 146 requests of ~13 s. Segments still start and end at detected silences, so no word is cut. The ASR operator carries the PCM inside the `SpeechRange` item (bounded channels keep at most ~8 items, 30 MB, in flight) instead of re-decoding per range.

- 2026-09-11 (GCP): Whisper's batched decoder returns one segment per 30 s piece, so the ASR operator splits segments longer than `models.asr.span_secs` (15 s) on word timings, cutting after sentence-final punctuation in the second half of a piece or else at the longest pause, then groups short segments as `subtitle_import` does. Word timings are stored as `{"w", "s", "e", "p"}` with absolute seconds. Each ASR request writes its own `Provenance` row (provider, model, media seconds, latency, attempts) so cost roll-ups work per request.
- 2026-09-11 (GCP): `subtitle_import` skips a sidecar whose cue text is identical to one already imported (yt-dlp writes YouTube auto-captions twice, as `en` and `en-orig`), and the rolling-caption collapse now also recognises a cue that repeats the second line of the previous two-line cue, which is how yt-dlp's current SRT conversion rolls. Before this each auto-caption sentence appeared four times in the index.
- 2026-09-11 (GCP): Provider-backed operators declare `required_roles()`; the scheduler checks the bindings at plan time and fails with exit code 8 naming the role, so an unconfigured provider is reported before any decoding. `config/gcp-a100.toml` is the config for this machine (Whisper on `127.0.0.1:9000`, ONNX models under `/data/videoindex/models`, a `coarse_asr` policy); pass it with `--config` or `VI_CONFIG`.
- 2026-09-11 (GCP): `shot_boundary` compares consecutive *sampled* frames (1 fps), so its shots are runs of visually continuous samples and a cut is placed at the first sample of the new shot; at that rate a shot is never shorter than the sampling interval, which is what slide-based lectures need. The distance is $0.5 \times (1 - \text{HSV histogram intersection, } 8 \times 4 \times 4 \text{ bins}) + 0.5 \times \text{edge change ratio}$ on a 96-pixel-wide grey image with one pixel of dilation tolerance; a cut needs the distance above 0.25 and either above 0.7 or above `mean + 3σ` and $2.5 \times$ the mean of the last 20 within-shot distances. The segments start at 0 and end at the video duration so the `shot` level has no gaps, and each carries its first sample as `keyframe_sample_id`. The 2-minute fixture (a hard cut every 10 s with a burned-in timer) yields exactly 12 shots, asserted in the pipeline tests.
- 2026-09-11 (GCP): Shot detection does not try to catch text-only slide changes. On the AI Engineer VC talk (white slides, small speaker inset, sponsor logos swapping every 10 s) a title change moves 2 to 9% of the pixels of the small grey image and about 10% of the edges, the same as a logo swap, so no threshold separates them; the 34-minute talk is six shots. Slide changes are detected from OCR text instead: the `ocr` operator runs on frames whose pixel-change measure (`FrameSignature::pixel_change`) fires, not only on pHash-distinct frames (a pHash of a mostly white slide ignores its text), and consecutive identical text collapses. `cargo run -p vi-perceive --example shot_debug -- <file> [t0] [t1]` prints the per-sample metrics for tuning.
- 2026-09-11 (GCP): The vector store is a flat, append-only, memory-mapped file pair per model (`vectors/<model>.vec` with normalised f32 rows, `<model>.meta` with embedding id, video id, target kind and an alive flag per row) searched exactly by brute force on rayon, not Lance. `lance` 11 pulls 689 crates (Arrow, DataFusion, an object-store stack) into a workspace of about 250 for a feature that at M1 to M4 scale (hundreds of thousands of vectors) runs in single-digit milliseconds with exact results, which the eval harness prefers over approximate ones. Deletes flip the alive flag, `compact` rewrites the files and updates the `row` column added to `embeddings` in schema v2, and `put_embeddings` replaces an earlier vector for the same (`target`, `model`). `lance` or `usearch` remain a drop-in behind `Storage` when approximate search is needed. `04-data-model.md` is updated.
- 2026-09-11 (GCP): `image_embed` consumes `Hashed` items, which now carry the frame, and embeds only frames farther than `PHASH_DEDUP_DISTANCE` from the previously embedded one (47 of 2,057 frames on the VC talk, about a third on slide-heavy lectures). Pixels are copied out to an owned buffer before batching so decoder slots are released at once; SigLIP base on 6 CPU cores embeds a batch of 16 in about 4 s. `text_embed` embeds transcript and OCR spans with bge-small (32 per batch, 342 spans in 11.6 s of CPU); its `ocrSpan` input is optional so it runs in policies without `ocr`. Operators declare `optional_inputs()` for exactly this.
- 2026-09-11 (GCP): `vi search` is hybrid: per-kind BM25 lists, a text-vector list (bge-small, query prefixed with bge's retrieval instruction through `TextEmbedder::embed_query`) over transcript, OCR and

description rows, and an image-vector list (SigLIP text tower) over frame embeddings, fused with RRF ($k = 60$). Grouping units are scenes when they exist, else shots cut into pieces of about 60 s (so a 29-minute static shot does not swallow a whole talk), else chapters, else 60 s windows; the chapter title is still reported when the unit falls inside one. Each piece of evidence lists the lists that ranked it (`bm25:transcript`, `text_vec`, `image_vec`) so relevance can be debugged. Query embedders load lazily and the SigLIP towers load one at a time, so a text query pays for bge and the SigLIP text tower only (about 1.5 s of model loading in the one-shot CLI; the server keeps them resident).

- 2026-09-11 (GCP): `sample` writes each `FrameSample` row before emitting the frame. Batching rows raced `image_embed`, whose embeddings reference the sample id: the vector's video could not be resolved and search then grouped it under a nil video. A per-frame insert is a few thousand statements per hour of video.
- 2026-09-11 (GCP): OCR is RapidOCR in-process: the PP-OCRv4 detector (DB, 960 px longest side, threshold 0.3, box score 0.5, unclip 1.6, axis-aligned boxes from connected components) and the English PP-OCRv3 recogniser (CTC, height 48, batches of 8 sorted by aspect ratio, `en_dict.txt` + blank + space = 97 classes). The `ocr` operator reads a frame when its pHash moved past the dedup distance, or its pixel-change measure against the last frame read is at least `models.ocr.min_pixel_change` (2%), or `max_gap_secs` (30 s) passed; lines already present in the previous read are not stored again. On a 47-minute slide lecture that read 247 of 2,815 frames and stored 841 lines, inside the decode's shadow. Reading order is rows by centre then left to right; boxes are stored normalised.
- 2026-09-11 (GCP): The operator output cache is a marker file per completed stage under `<index>/cache/operators/`, keyed by content hash, operator id and version, and the operator's `cache_params()` (provider, model, thresholds, sampling rate). The outputs themselves are the rows in the index (videos are keyed by content hash, so identical content already shares them). At plan time each stage is *run*, *replayed* or *skipped*: skipped when cached and no running consumer needs its items; replayed (`Operator::replay`, re-emitting stored spans or segments without recomputing: `subtitle_import`, `asr`, `ocr`, `shot_boundary`) when cached but a consumer runs; run otherwise, and always run when one of its producers runs, because a re-run producer replaces the rows the stage's outputs hang off. This replaces M0's all-or-nothing resume: adding `ocr` to a policy after a `coarse_asr` run re-decodes (sampling cannot replay) but leaves ASR and VAD untouched, and `text_embed` gets its transcript spans replayed. `--force` ignores markers; a stage that skipped or failed inputs leaves no marker so the next run completes it.
- 2026-09-11 (GCP): Budgets: `Budget::for_policy` turns `max_cost_usd_per_hour` and `max_wallclock_per_hour` (now parsed: 20m, 1h30m, 90s; 0 means unlimited) into a per-job ceiling using the video's duration (at least a minute's worth). Provider-backed operators call `allow_provider_call()` before each call and record spend from `CallStats`; once a limit is hit no further calls are issued, the inputs are counted as skipped, the stage completes, and the report says which limit stopped it. Provider errors after the adapter's retries are recorded per input range (first 50 kept with times and messages) and the stage continues; a stage whose every input failed is marked failed and the video ends `failed`.
- 2026-09-11 (GCP): `Http` and `ObjectStore` acquirers download into `<cache_dir>/incoming/<8-char URL hash>-<file name>` and then reuse `LocalFile`, so all four acquirers produce identical index contents and sidecars next to a remote object (`.info.json`, `.srt`) are picked up. `Http` claims only URLs whose path ends in a media extension (video-site URLs stay with `YtDlp`), disables automatic redirects and resolves every hop's host itself, refusing private, loopback, link-local, CGNAT and metadata addresses unless `media.download.allow_private_addresses` is set; it resumes with `Range`, enforces `max_bytes` from `Content-Length` / `Content-Range` and while streaming, and rejects `text/html` bodies. `ObjectStore` uses the `object_store` crate with credentials from the environment; `r2://` is S3 with `media.download.r2_endpoint` or `R2_ENDPOINT`; a URL ending in `/` expands to the media objects under the prefix. Tests run against a local HTTP stand-in and `object_store`'s local filesystem store.

- 2026-09-11 (GCP): `default_policy` defaults to `auto: coarse_only` (every coarse operator: subtitles, VAD, ASR, sampling, shots, pHash, thumbnails, SigLIP, OCR, bge) when the `asr`, `ocr`, `image_embed` and `text_embed` roles are all bound, else `coarse_local`. `lecture_default` keeps the fine pass (`scenes`, `chapters`, `vlm_describe`, `entities_events`) and fails at plan time until M2 implements them; `text_embed` moved into the coarse pass because hybrid search needs span vectors before any VLM runs.
- 2026-09-11 (GCP): No sort comparator may use a tolerance or `partial_cmp(..).unwrap_or(Equal)`: Rust's sort panics on comparators that are not total orders, and the OCR reading-order sort (rows matched within half a box height) did exactly that on the first long workshop, killing the rayon task and failing every OCR read of that video. Reading order now sorts by centre y, groups rows greedily, and sorts each row by x; every float sort uses `total_cmp`.
- 2026-09-11 (GCP): `dataset/devset.jsonl` holds the retrieval dev set: 72 questions over all 30 dataset videos (65 transcript, 3 OCR, 4 visual). Transcript and OCR questions carry an `anchor` phrase that `scripts/devset_eval.py` resolves to a time from the video's captions (or the index's OCR spans), so ground truth follows the media rather than hand-typed seconds; visual questions carry `t0/t1` from contact sheets. A hit counts when the result's video matches and its range, widened by 30 s, overlaps the anchor's range (cue time -5 s to +30 s). The questions are paraphrases, not caption text, so BM25 alone misses many of them and the vector lists have to earn their place.
- 2026-09-11 (GCP, M2): Chat adapters share one SSE parser (`vi_providers::sse`, also accepts JSON lines) and stream `GenerateEvent`s from `futures::stream::unfold` state machines, so tool-call fragments are assembled per adapter (OpenAI by index, Anthropic by content block, Gemini whole per part) and every stream ends with `Usage` then `Done { finish_reason, stats }` with `finish_reason` normalised to `stop`, `length` or `tool_use`. Images go in as base64 JPEG (quality 85) from raw RGB; only Gemini takes video inline (under 19 MB; file upload is deferred). `ContentPart::ToolCall / ToolResult` carry the tool name because Gemini keys function responses by name, not id. Default prices per model family live in `vi_providers::pricing` and are overridable per provider; they are estimates to check against vendor lists. Anthropic rejects `temperature` on Claude 5 models, so it is sent only when non-zero; a live call to `claude-sonnet-5` with one tool costs about \$0.003 and 2 s.
- 2026-09-11 (GCP, M2): `vi-agent` runs the loop from `06-query-and-agents.md` around the `agent_llm / agent_vlm` roles: a system prompt (versioned in `vi-providers/prompts/agent_system.md`, hash recorded) plus a compact list of the index's videos (id, duration, title) when there are at most 40, then a tool-using generation loop. Tools: `search`, `list_videos`, `timeline`, `get_transcript`, `get_ocr`, `get_descriptions`, `view`, and `describe` when a `vlm_describe` role is bound. Tool results are JSON text; a `view` grid follows as an image in a user turn because OpenAI-shaped APIs cannot carry images in tool messages. Citations are inline markers `[[cite:VIDEO_ID:T0-T1]]` that a streaming scanner removes from the token stream and emits as `Citation` events typed by the evidence the loop has seen (`transcript`, `ocr`, `frame`, `description`, else `range`); markers naming unknown videos are dropped. Budgets (`max_tokens`, `max_cost_usd`, `max_wallclock_secs`, `max_tool_calls`) are checked before each provider call; when one is hit the model is asked to answer from what it has and `Done { partial: true, reason }` says which limit. Sessions keep the last 20 question/answer pairs in the `sessions` table with a 24 h TTL. `Policy` decides tools for baselines (`RetrievalOnlyPolicy`: one search, then answer); the LLM still writes the answer. First live run on the VC talk: 3 tool calls, correct timestamps, \$0.065, 14.6 s with `claude-sonnet-5`.
- 2026-09-11 (GCP, M2): Frame grids are composed in `vi-perceive::grid` with a built-in 5×7 bitmap font for the HH:MM:SS labels (no font files or text-rendering crates), nearest-neighbour tiles, `view` clamps to 120 s and 16 frames (lowering fps), and grids are stored as PNG blobs so a citation of a `view` can be rendered later. The first run rendered 9 frames of the VC talk in 1.8 s including decode.
- 2026-09-11 (GCP, M2): The fine pass. `scenes` groups shots with a boundary strength of `1 - cosine(keyframe SigLIP embeddings)` halved when a transcript span crosses the boundary, merging across weak boundaries (< 0.5) and any scene under 20 s while staying under 180 s; shots get `parent_id`. `chapters` keeps imported chapters when the source had them; otherwise it embeds each

scene's transcript with the `text_embed` role, computes TextTiling-style depth scores over the scene sequence, adds a bonus where the first title-like OCR line changes, and picks boundaries for about one chapter per 10 minutes with a 3-minute minimum, titling each from its first title-like OCR line.

`vlm_describe` sends a 3×3 grid of a scene (fps chosen to fit nine frames) plus the scene transcript as data to the `vlm_describe` role with the versioned prompt, asks for JSON when the provider supports schemas, stores a `Structured Description` (flattened text for FTS and embeddings, JSON kept) and writes the summary onto the scene; the grid is kept as a blob. `entities_events` sends 5-minute windows of descriptions and transcript to the `extract_llm` role and stores entities (canonicalised by lower-cased name) with mentions and events; a re-run replaces a video's extractions. All four honour the job budget and record per-range failures. A job whose policy has fine operators ends in state `fine`.

- 2026-09-11 (GCP, M2): The Python binding (`bindings/python`, PyO3 0.29 + maturin, abi3 for Python 3.10+) owns one tokio runtime and releases the GIL (`Python::detach`) around every core call. `Index.add` returns a `Job` whose `progress()` iterates the event bus and `wait()` returns the reports; `search`, `timeline`, `videos`, `status` return plain dicts (JSON through `serde`, `{num, den}` timestamps flattened to seconds); `ask` returns an `AskStream` usable with `for` or `async for` (`pyo3-async-runtimes`) or drained with `collect()`. `frame` returns a NumPy `HxWx3 uint8` array as a copy of the decoded buffer; the zero-copy buffer-protocol view over `Arc<FrameBuffer>` in `03-rust-boundary.md` is deferred until a caller needs it (a 640 px frame copies in microseconds). Custom operators and policies from Python are not wired yet. The decode worker cannot be the Python process, so `media.worker.path` must point at a `vi` or `vi-media-worker` binary; the wheel will bundle the worker later. Tests: `pytest bindings/python/tests` after `maturin develop`.
- 2026-09-11 (GCP, M2): First full-pass run on the VC talk found two defects: a 29-minute static shot became one scene (scenes now cut shots longer than 180 s into pieces of about 120 s, snapped to transcript span starts), and the extraction model wrapped its JSON in prose or ran out of tokens (the first `{...}` object is now extracted and the budget is 4,000 tokens). Stages that store without emitting (embeddings, extraction) are no longer judged failed when one batch fails: the all-failed rule counts stored rows too.
- 2026-09-11 (GCP): The decode worker is spawned from `/proc/self/exe` on Linux, not from `current_exe()`'s path. Rebuilding `vi` while a long index job ran replaced the binary on disk; every later worker spawn failed with "failed to spawn .../vi" and 25 of 30 videos errored in seconds. `/proc/self/exe` stays valid for the life of the process. The same run also showed that the design's `max_wallclock_per_hour = "20m"` is too tight for a CPU-only ONNX pass on a loaded machine (a 1 h 46 min workshop hit it and skipped 78 OCR/embedding inputs), so the built-in policies default to 60 min and the GCP config removes the limit for the dataset run.
- 2026-09-11 (GCP, M2): The first fine pass on the VC talk (17 scenes of about 2 minutes, 12 structured descriptions, 3 topic chapters) titled chapters from sponsor logos ("AI Engineer", "Microsoft") because any OCR line of 8+ characters counted as a title. A title now needs at least three words of mostly letters, no URL or e-mail, and a box in the upper 60% of the frame at least 3% high. Fine-pass cost on that talk with `claude-sonnet-5`: descriptions \$0.34 (15 calls), extraction \$0.52 (12 five-minute windows), about \$1.6 per hour of video against the \$2 default ceiling.
- 2026-09-11 (GCP): CUDA 13.1 toolkit and cuDNN 9 are installed from NVIDIA's `ubuntu2404 apt` repository (the Ubuntu 26.04 package is CUDA 12.4, and `ort 2.0.0-rc.13` ships ONNX Runtime 1.28 binaries built only for CUDA 13). `vi` is built with `--features cuda`; `models.device = "auto"` then resolves to `cuda` and `vi doctor` shows the toolkit and libraries. The first GPU run found that `onnx_local` handed every `providers.ocr()` / `image_embedder()` call a fresh handle whose model cell was empty, so the models were loaded 518 times in one video (1.2 s each on CUDA; also true, if cheaper, on the CPU). The loaded models now sit behind shared `Arc<OnceLock>`s and load once per process.
- 2026-09-11 (GCP): No hardware video decode on this machine. The driver install has no `libnvcuvid.so`, and the dataset is 22 AV1 + 8 VP9 streams; the A100's NVDEC has no AV1 support, so

even a working `libnvcuvid` would cover a quarter of the corpus. Decode stays on the CPU (`libdav1d` / `libvpx` through `libav`) and `vi-media` remains the pass's bottleneck: with the ONNX models on the GPU (10 to 35× faster per call, see `MACHINE.md`) every other coarse stage finishes in the decoder's shadow. `onnx_bench` (an example in `vi-perceive`) is the per-call benchmark used for such comparisons.

- 2026-09-12: Python operators and policies. Custom operators plug in through `Scheduler::register_operator(name, factory)`; a factory builds a fresh operator per job so per-job state never crosses jobs. A custom operator with a built-in name replaces the built-in one (that is how an experiment swaps a stage). Items cross the boundary as JSON dicts plus NumPy arrays for pixels and audio; rows come back as JSON and `vi_pipeline::callback::store_rows` turns them into stored rows (ids, provenance, timestamps at a 1 MHz timebase) before emitting them, so bindings never touch storage themselves. A Python operator's outputs are cached under its `id`, `version` and `params` like any stage, and its `run` executes on a tokio blocking thread holding the GIL only for the call. Operator ids are `&'static str` in the trait; a registered Python id is leaked once, which is fine for a handful of registrations per process. Inline policies (`add(..., policy={...})`) default unset operator lists to empty rather than `lecture_default`, so a dict listing only `coarse` does not silently pull in the VLM stages. A Python policy returns `{"tool", "args"}` or `None`; the LLM still writes the final answer.
- 2026-09-12: Full-text queries OR their terms. `fts_query` used to emit every word as a required term (FTS5's implicit AND), which is right for keyword lookups and wrong for the questions people type: on the 72-question dev set the BM25 lists were empty for almost every miss and the fusion ran on the vector lists alone. Terms are now OR-ed after dropping English stopwords (a query made only of stopwords keeps them), the last term still prefix-matches, and BM25 ranks rows matching more terms higher. The eval's caption anchors now match across up to three consecutive cues, since captions break sentences mid-phrase; eight questions were unscorable before.
- 2026-09-12: cuDNN algorithm search set to heuristic, OCR recogniser widths bucketed. The "decode-bound" reading of the first GPU run was wrong: decode alone is 39.5 s for the 47-minute test lecture while the visual pass took 112 s (CPU) and 177 s (first CUDA build). The CUDA loss was cuDNN's exhaustive per-shape convolution search on the OCR recogniser, whose padded batch width changed nearly every call. `OnnxSession` now builds the CUDA provider with `ConvAlgorithmSearch::Heuristic`, and `RapidOcr::recognize` rounds the batch width up to a multiple of 32 (a few dozen shapes per video instead of hundreds; 2% fewer recognised lines on the test video from the extra zero padding, within the confidence filter's noise). Visual pass: 50 s on the GPU, 10 s above the decode floor. The `onnx_bench` example measures a single fixed shape and cannot see this class of problem; shape-varying workloads need the pipeline timing.

Development machine

Recorded 2026-09-11 at the M1 kickoff on the GCP A100 machine. `vi doctor` reports the live values; this file is the snapshot the M1 decisions were made against. The previous host is described in [MACHINE-azuremc.md](#).

Hardware

Item	Value
Hostname	<code>instance-20260911-162245</code> , GCP <code>a2-ultragpu-1g</code> , zone <code>us-central1-c</code>
OS	Ubuntu 26.04.1 LTS, kernel 7.0.0-1011-gcp, x86_64
CPU	Intel Xeon @ 2.20 GHz (Cascade Lake class), 12 vCPUs (6 cores × 2 threads), AVX-512
RAM	167 GiB total, 164 GiB available, no swap
GPU	1 × NVIDIA A100-SXM4-80GB (80 GB, not the 40 GB the kickoff assumed for <code>a2-highgpu</code>). Driver 595.91.07, CUDA 13.2 driver API. CUDA toolkit 13.1 and cuDNN 9 installed on 2026-09-11 from NVIDIA's <code>ubuntu2404</code> apt repository; see "GPU stack" below.

The design docs assume an 8-core server; this machine has 12 vCPUs, so timings here are above the design baseline but well below the 24 vCPUs recommended in `MACHINE-RECOMMENDATIONS.md` .

Disks

Device	Size	Free	Mount	Notes
<code>/dev/sda1</code>	750 G	715 G	<code>/</code>	OS persistent disk (ext4). The repository and the Rust <code>target/</code> live here.
<code>/dev/sdb</code>	2 T	1.9 T	<code>/data</code>	Persistent disk (ext4, discard). <code>/data/videoindex</code> lives here: <code>videos/</code> (media cache, <code>incoming/</code> , <code>fixtures/</code>), <code>indexes/</code> , <code>models/</code> , <code>asr/</code> (Python venv), <code>logs/</code> .
<code>/dev/nvme0n1</code>	375 G	–	not mounted	Local SSD, blank (no filesystem). The kickoff planned to put <code>target/</code> , <code>/dev/shm</code> -style frame slots and ffmpeg temp files here; formatting it was not done in the kickoff session (see <code>DECISIONS.md</code>). <code>/dev/shm</code> is a 84 G tmpfs, far above the 4 G the frame slots need, so nothing is lost by leaving it unused.

`/data/videoindex/videos/incoming/` holds the two dataset playlists transferred from the Mac: **30 MP4 files, 5.1 GB, 36.6 hours**, every one with its `.info.json` and 2 to 4 `.srt` sidecars (`PLcfpQ4tk2k0XfT3YLdjLXKHAaBJ9MX6re` AI Engineer workshops, 19 files; `PLS01nW3RtgoqGkm4UeqNeZLccW-0Gc1fJ` Berkeley Agentic AI MOOC, 11 files) plus `yt-dlp's` `archive.txt` .

Toolchain

Tool	At kickoff	Now
<code>rustc</code> / <code>cargo</code> / <code>rustup</code>	missing	installed via <code>rustup</code> , stable 1.98.1 (<code>~/ .cargo/bin</code>)
<code>cargo-deny</code>	missing	installed via <code>cargo install</code>
<code>gcc</code> , <code>cmake</code> , <code>pkg-config</code> , <code>clang</code> 21, <code>libclang-dev</code>	missing	installed (<code>build-essential</code> , <code>cmake</code> 4.2)

Tool	At kickoff	Now
ffmpeg / ffprobe	missing	installed 8.0.1 with dev headers: libavformat 62.3, libavcodec 62, libswscale, libswresample, libavfilter, libavdevice. ffmpeg-next 9.0 built against it without changes.
libsqlite3-dev , fonts-dejavu-core	missing	installed
python3	3.14.4 (system)	unchanged. Too new for the ML wheels, so the ASR server runs in a uv-managed Python 3.12 venv at /data/videoindex/asr/.venv
uv	missing	installed in ~/.local/bin
node / npm	missing	not installed (M3)
yt-dlp	missing	installed standalone binary 2026.08.19 in /usr/local/bin
docker , nvidia- container-toolkit	missing	not installed . Adding Docker is a system-service change; the ASR server runs without it (see below).
caddy , nginx	missing	not installed on purpose: this is the indexing box, serving is a separate machine
git	2.53.0	unchanged; gh missing

GPU stack

- **nvidia-smi** works; the driver supports CUDA 13.2. The machine came without a toolkit. `cuda-toolkit-13-1`, `libcudnn9-cuda-13` and `libcudnn9-dev-cuda-13` were installed from NVIDIA's `ubuntu2404` repository (`cuda-keyring` ; Ubuntu 26.04's own package is CUDA 12.4, which the ONNX Runtime binaries below cannot use). `ldconfig` now lists `libcudart.so.13`, `libcublas.so.13`, `libcudnn.so.9`, `libcufft`, `libcublas` and `libnVRTC`; `nvcc` is at `/usr/local/cuda-13.1/bin/nvcc`.
- **ASR**: faster-whisper 1.2.1 on CTranslate2 4.8.2 in the Python 3.12 venv, with `nvidia-cublas-cu12` and `nvidia-cudnn-cu12` pip wheels supplying the CUDA 12 runtime libraries in user space (`scripts/asr_server.sh` puts their `lib/` directories on `LD_LIBRARY_PATH`; CTranslate2 does not find them by itself). `ctranslate2.get_cuda_device_count()` reports 1. First measurement: a 60 s lecture clip transcribes in 1.46 s with word timestamps (41× real time, batch 16, beam 5, fp16), 4 GB of VRAM in use. In the pipeline (`coarse_asr` policy, 4 concurrent requests of up to 120 s, server batch 16): a 34-minute talk with 31.7 minutes of speech transcribed in 65 s, 29× real time; the whole coarse pass with subtitles, VAD, ASR, sampling, pHash and thumbnails took 81 s and was bounded by the video decode (sample stage 81 s). The server (`scripts/asr-server/server.py`, started by `scripts/asr_server.sh`) exposes the OpenAI `/v1/audio/transcriptions` API on `127.0.0.1:9000` with `large-v3` in fp16. Model weights cache in `/data/videoindex/models/whisper/`.
- **ONNX Runtime** (VAD, SigLIP, RapidOCR, bge-small): `ort 2.0.0-rc.13` downloads ONNX Runtime 1.28 built for CUDA 13 when `vi` is compiled with `cargo build --release -p vi-cli --features cuda`; the build drops `libonnxruntime_providers_cuda.so` and `libonnxruntime_providers_shared.so` next to the binary and `models.device = "auto"` then resolves to `cuda` (`vi doctor` prints `device cuda` (config: `auto`, `cuda` feature on)). A CPU-only build (`cargo build --release`) still works and is what CI produces. The first GPU run of the `visual` policy on the 47-minute slide lecture took 473 s against 160 s on the CPU and exposed a bug rather than a slow GPU: the adapter handed every operator call a fresh handle with an empty model cell, so the 518 OCR/embedding calls each re-created a CUDA session (about 1.2 s each; the CPU build paid the same reload at a few tens of milliseconds). With the models shared behind `Arc<OnceLock>` (one `loaded ONNX model` line per model) the same run took 220 s while the 30-video dataset run was saturating the CPUs (load average 9 on 12 vCPUs), so a clean GPU-versus-CPU number waits for an idle machine; see the timing table below once it exists.

Per-call latency of the models themselves (`cargo build --release -p vi-perceive --features cuda --example onnx_bench` , then `onnx_bench /data/videoindex/models <cpu|cuda> frame.png ; 640×360` slide frame, 20 iterations, measured while the dataset run had the CPUs at load 9):

Call	CPU (6 threads)	CUDA	Speed-up
SigLIP image embed, batch 1	270 ms	11.7 ms	23×
SigLIP image embed, batch 8	1623 ms	46 ms	35×
SigLIP text embed (one query)	83 ms	6.9 ms	12×
bge-small, 32 spans	165 ms	8.3 ms	20×
RapidOCR detect + recognise (4 lines)	262 ms	26 ms	10×
Model load (each)	0.07 to 0.47 s	0.10 to 0.52 s	

That made the models look free and the pass decode-bound, which the idle-machine measurements below contradicted: decode alone (`m0` : sample, pHash, thumbnails) takes 39.5 s for this 47-minute video (71× real time), yet the `visual` pass took 112 s on the CPU build and 177 s on the first CUDA build. OCR alone was 96 s (CPU) against 176 s (CUDA); image embedding alone 53 s against 38 s. The CUDA OCR loss was cuDNN's default *exhaustive* convolution-algorithm search, which re-benchmarks every convolution for each new input shape, and the recogniser's batch tensor had a new width on almost every call. With the heuristic search (`cudaconv_algo_search`) and recogniser widths rounded up to multiples of 32, the same video runs:

Policy (idle machine, this 47-min 720p VP9 lecture)	CPU build	CUDA build (first)	CUDA build (fixed)
<code>m0</code> : decode + pHash + thumbnails	39.7 s	39.5 s	
image embedding only (sample, phash, image_embed)	52.9 s	38.4 s	
OCR only (sample, phash, ocr)	95.9 s	175.9 s	48.2 s
<code>visual</code> (all of the above + shots + text_embed)	112.3 s	177.4 s	50.2 s

So with the GPU the visual pass runs at 56× real time and sits about 10 s above the decode floor; on the CPU it is OCR-bound at 25×. The 30-video dataset run (`coarse_only` , CPU build, loaded machine) averaged 7× real time with OCR-heavy slide decks at 5×; the same run with the fixed CUDA build should be bounded by decode plus ASR. - **Hardware decode is not available here.** `ffmpeg` 8 has the `cuda` hwaccel and the `*_cuid` decoders compiled in, but the GCP driver install ships no `libnvcuvid.so` , so `-hwaccel cuda` fails at setup (`Failed setup for format cuda`) and `vp9_cuid` exits immediately. It would not help much anyway: the 30 dataset videos are 22 AV1 and 8 VP9 streams at 720p, and the A100's NVDEC (GA100) decodes H.264, HEVC and VP9 but not AV1. Software decode of the 720p VP9 lecture runs at 31× real time in plain `ffmpeg` (9.5 s for 5 minutes, 2.7 cores); `vi-media` 's worker with 1 fps sampling, 640 px scaling and shared-memory delivery reached about 18× on an idle machine (160 s for 47 minutes). The decode itself is not the coarse pass's bottleneck (see the table above); OCR was. - vLLM for the open-VLM A/B runs (M2/M4) is not installed. With 80 GB, a 32B VLM in fp16 fits on the single GPU, so no tensor parallelism or 4-bit quantisation is needed.

Model files

Under `/data/videoindex/models/` (934 MB before Whisper):

Directory	Files	Source
<code>silero-vad/</code>	<code>silero_vad.onnx</code> (2.2 MB), MIT licence	<code>onnx-community/silero-vad</code>
<code>siglip-base-patch16-224/</code>	<code>vision_model.onnx</code> (372 MB), <code>text_model.onnx</code> (441 MB), <code>tokenizer.json</code> , configs	<code>Xenova/siglip-base-patch16-224</code> (768-dim, 224 px; the <code>so400m-384</code> export does not exist on the Hub)

Directory	Files	Source
bge-small-en-v1.5/	model.onnx (133 MB), tokenizer.json	Xenova/bge-small-en-v1.5 (384-dim)
rapidocr/	ch_PP-0CRv4_det_infer.onnx , ch_PP-0CRv4_rec_infer.onnx , en_PP-0CRv3_det_infer.onnx , en_PP-0CRv3_rec_infer.onnx , ppocr_keys_v1.txt , en_dict.txt	SWHL/RapidOCR , PaddleOCR dictionaries
whisper/	faster-whisper large-v3 (CTranslate2, ~3 GB)	downloaded by the ASR server on first start

Network

- **YouTube via yt-dlp: blocked**, as on azuremc (datacenter IP). Videos arrive by rsync into `incoming/` ; they are already there.
- crates.io, GitHub, the Ubuntu archive, Hugging Face and PyPI are reachable.

Existing deployment state

- `/etc/videoindex` : does not exist. No `videoindex-*` systemd units, no `videoindex` user.
- Listening ports at kickoff: only `sshd` (22) and `systemd-resolved` (53 on loopback). No `nginx`, no `Docker`. The ASR server adds `127.0.0.1:9000` (loopback only).
- DNS: `videoindex.app` and `www.videoindex.app` resolve through Cloudflare (`104.21.65.27` , `172.67.139.223` , and the AAAA records recorded for azuremc); `app.` and `api.videoindex.app` have no records. This machine is not a serving host and no reverse proxy is installed here.
- User `nash` has passwordless `sudo` (`google-sudoers`).

M0 re-verification on this machine (2026-09-11, release build)

`cargo build --release` && `cargo test --workspace` pass unchanged against `ffmpeg 8.0.1` and `rustc 1.98.1` (azuremc had `ffmpeg 6.1`). `cargo clippy -D warnings` and `cargo deny check` pass.

`vi index /data/videoindex/indexes/m0-timing.vidx /data/videoindex/videos/fixtures/synthetic-1h-720p.mp4 --policy m0 --compact` on the regenerated 1-hour 720p H.264 30 fps fixture (policy `m0` : sample at 1 fps, pHash, WebP thumbnails at 320 px):

Metric	azuremc (4 vCPUs)	this machine (12 vCPUs)
Wall clock, whole job	167.6 s	48.6 s (3.4× faster)
Acquire (blake3) + probe	0.9 s	0.6 s
CPU time	418 s user + 11 s sys (255%)	313 s user + 7 s sys (657% of one CPU)
Frames delivered	3,600	3,600
Peak RSS, parent	90 MB	117 MB
Index size	14.6 MB	14.5 MB (1.0 MB SQLite, 13.5 MB in 3,600 WebP blobs)

The fixture regenerated here is 917 MB (azuremc: 920 MB; same script, `ffmpeg 8` instead of `6`). Total CPU time fell from 429 s to 320 s (newer `libavcodec`, `AVX-512`), and the job used 6.6 of the 12 vCPUs, so with two or three jobs in parallel the machine is saturated. At this rate the 36.6 hours in `incoming/` take about 30 minutes for the M0 stages alone, before ASR, embeddings and OCR.

Coarse pass with every M1 operator on the CPU (`coarse_asr` / `visual` policies: sampling, pHash, thumbnails, shots, SigLIP embeddings of pHash-distinct frames, RapidOCR on changed frames, bge embeddings of spans, plus VAD and Whisper on the GPU): 34-minute talk in 91 s, 47-minute slide lecture

(no ASR) in 160 s, both bounded by the H.264 decode with the other stages in its shadow. About 3.3 minutes per hour of video; the 36.6 hours in `incoming/` are roughly two hours of indexing on this machine, one at a time.

`vi doctor` on this machine reports the GPU line as `NVIDIA A100-SXM4-80GB, 81920 MiB, 595.91.07` followed by `count 1 driver 595.91.07 cuda (driver) 13.2 cuda toolkit 13.1 cuda libs: libcudart.so libcublas.so libcudnn.so` (before the toolkit install: `cuda toolkit missing cuda libs: none`). It was extended in this kickoff to print the GPU count, the CUDA version the driver supports, the toolkit version when `nvcc` exists, and which CUDA runtime libraries the dynamic linker can find; it also counts incoming videos recursively (playlists are directories) and finds `rustc / cargo` in `~/ .cargo/bin` when they are not on `PATH`.